Kafka replication protocol

Author: Jack Vanlightly

This document is a description of the Kafka replication protocol with KIP-966. The objective is to provide a description that is similar to the descriptions of other protocols such as Raft, Multi-Paxos and Viewstamped Replication Revisited. This is not a research paper, there are no novel approaches proposed - it is simply a description of how the Apache Kafka replication protocol works. Everything you read here is the product of an open-source community of engineers that has evolved Apache Kafka since its inception.

Contents

1. Introduction
2. The replication algorithm
3. Detecting and avoiding log divergence
4. Partition reassignment (reconfiguration)
5. Asynchronous storage and recovery
6. Replication correctness
7. Configuring for availability over consistency
8. Broker lifecycle

Glossary

- ISR: In-Sync Replicas.
- ELR: Eligible Leader Replicas.
- HWM: High watermark.
- LEO: Log End Offset.
- LESO: Leader Epoch Start Offset.
- Offset: The position of a record in the log.
- Complete replica: A replica that hosts the complete committed log.
- Unclean shutdown: An abrupt termination or one such that the full shutdown sequence could not execute.
- Unclean replica/broker: A broker (and its replicas) which has restarted after an unclean shutdown.
- MinISR: min.insync.replicas.

1. Introduction

Apache Kafka is a distributed log system which provides a streaming API for durable, partitioned and replicated logs. In this API, streams are exposed as topics which are themselves sharded into one or more partitions where each partition is an ordered, replicated log of immutable entries.

Each topic partition uses the Kafka replication protocol for its replication and high availability logic. The core protocol focuses on log replication, but can be extended to be a State Machine Replication protocol (with peers such as Raft and Multi-Paxos). The protocol is specific to Kafka and not a general purpose SMR protocol that has many implementations in the wild.

Kafka itself has many components implemented as replicated state machines (RSMs) built on top of the replication protocol. Kafka uses a Kafka-built-on-Kafka approach for many of its distributed components that require fault tolerance and high availability.

Examples of partition backed RSMs in a Kafka cluster:

- Transaction coordinators.
- Consumer group offset management.

Within Apache Kafka, the replication protocol should not be confused with either the Kafka wire protocol that the clients speak or KRaft which is a Raft-backed cluster control plane.

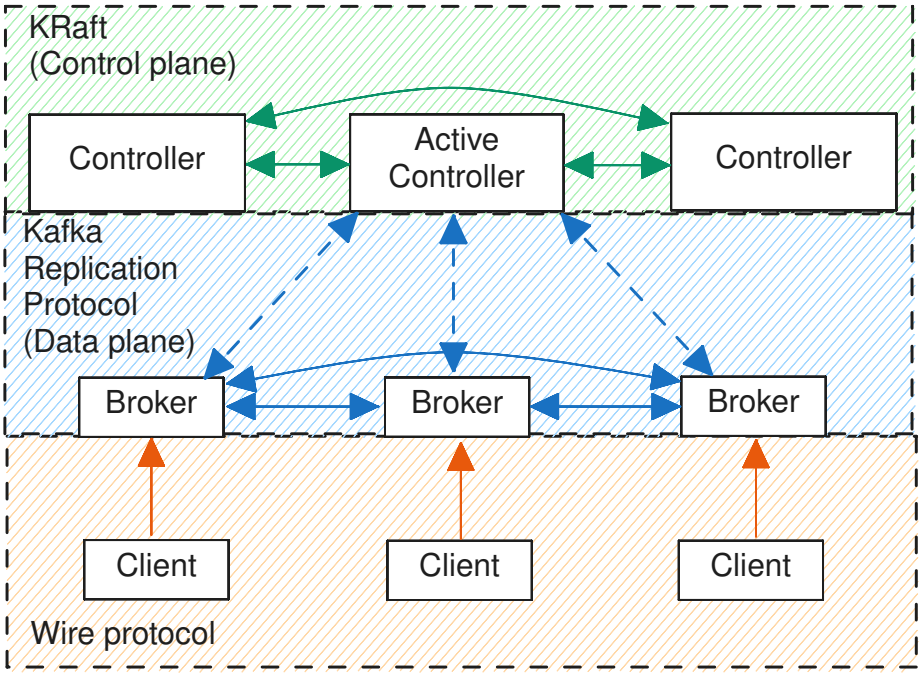

Fig 1. The Kafka protocols.

While a single topic may be composed of multiple partitions, the unit of replication is the partition. A single partition can be considered a “cluster” of nodes where a node is called a “replica”. Replicas are hosted on brokers but one broker can only host one replica of any given partition.

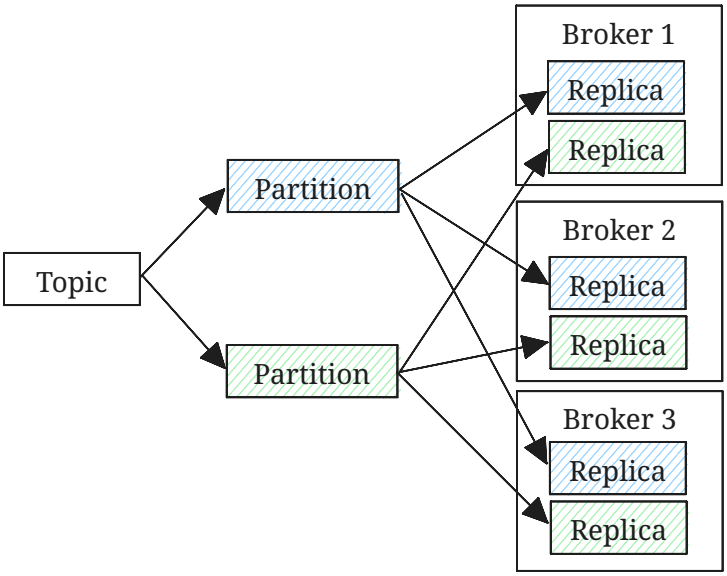

Fig 2. A topic composed of 2 partitions spread over 3 brokers.

The replication protocol works at the partition level and is responsible for:

- Replicating partition data between replicas.
- Maintaining partition replica leadership (the protocol is a leader-follower design).
- Providing linearizable semantics to clients.

1.1 KRaft not Apache ZooKeeper

This protocol description describes how the KRaft-based Kafka replication protocol works. The ZooKeeper version is slightly different and is not covered.

We should first disambiguate KRaft from the Kafka replication protocol.

Kafka has a very important sub-system, called the controller, which can be thought of as the cluster control plane. When you create a topic, delete a user, modify some configuration etc, that is executed by the controller. The old controller was backed by Apache ZooKeeper whereas the new controller is built on KRaft - Raft-based replicated state machine.

Where KRaft is the control plane, the Kafka replication protocol is the performance optimized data plane. Topic partitions speak the Kafka replication protocol to consistently replicate their data. It is not Raft based but does interact with the KRaft controller - ultimately the Kafka replication protocol is a collaboration between the controller and the partition replicas.

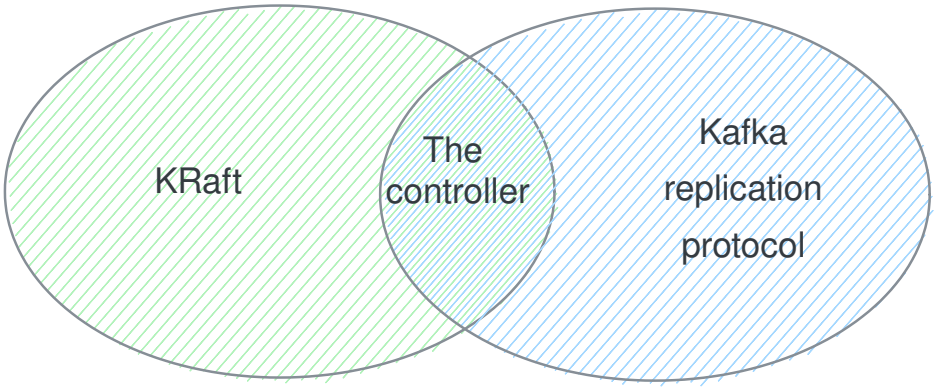

Fig 3. KRaft and the Kafka replication protocol insersect at the controller.

2. The replication algorithm

The replication algorithm is leader-based like other protocols such as Raft and Multi-Paxos. Unlike those protocols, Kafka replication is pull-based and the various functions of failure detection, leader election and replication are distributed across two node types: partition replicas and the controller.

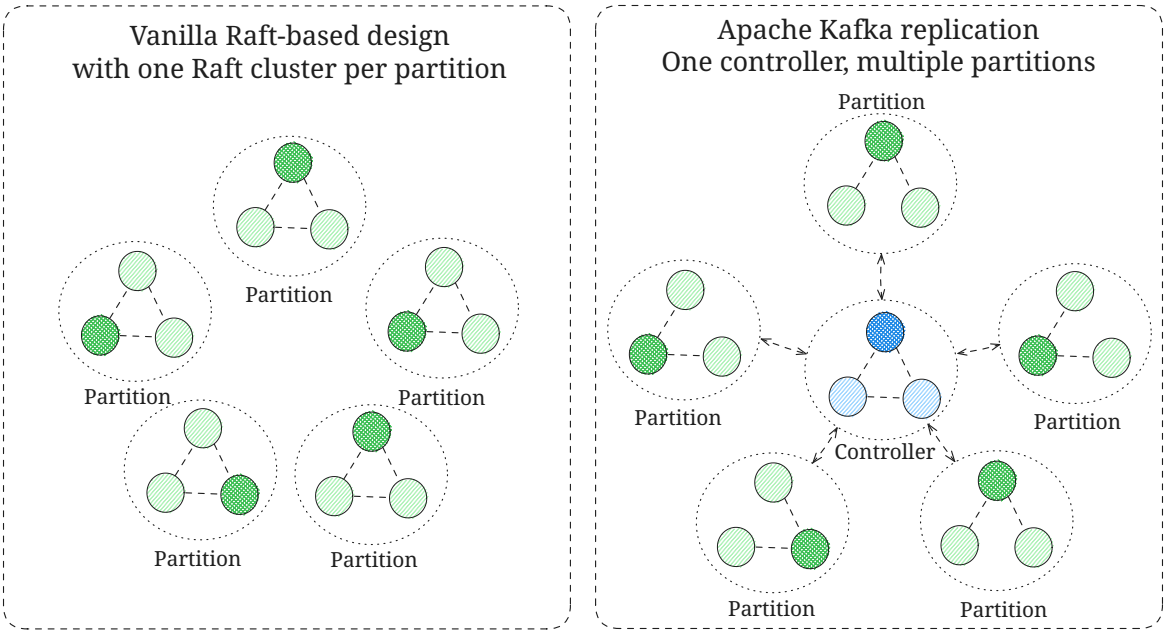

Fig 1. Comparing a design based on one Raft cluster per partition to a Kafka cluster where each partition forms its own replication unit but communicates with a shared controller.

1.2 Controllers, brokers and partitions

1.2.1 Metadata log replication

The controller is the cluster control-plane responsible for managing the cluster state, storing metadata and disemminating that metadata to brokers. It is a replicated state machine based on Raft and will have at least three nodes where one is the leader - known as the active controller.

With KRaft-mode, a Kafka cluster can be deployed with controllers and brokers as separate processes, or with the two combined in each node. This description will simply describe the logical topology with controllers and brokers as separate entities.

All cluster operations are performed on the active controller and usually involve a change to some metadata. Metadata can relate to users, permissions, configuration and partitions (among others). The controller has many sub-components for heartbeat handling, cluster changes and changes to partition metadata. All cluster metadata changes are first written to the controller Raft log and once committed are executed, changing the in-memory state of the controller. The metadata log is itself replicated to each broker with sequential consistency - that is to say that there is a total order (that matches the controller log) but each broker could be lagging behind the controller by some arbitrary amount. This metadata log copy is persistent and so data does not need to be re-fetched after a shutdown or crash. Each broker learns of the state of the cluster by receiving metadata updates in the log.

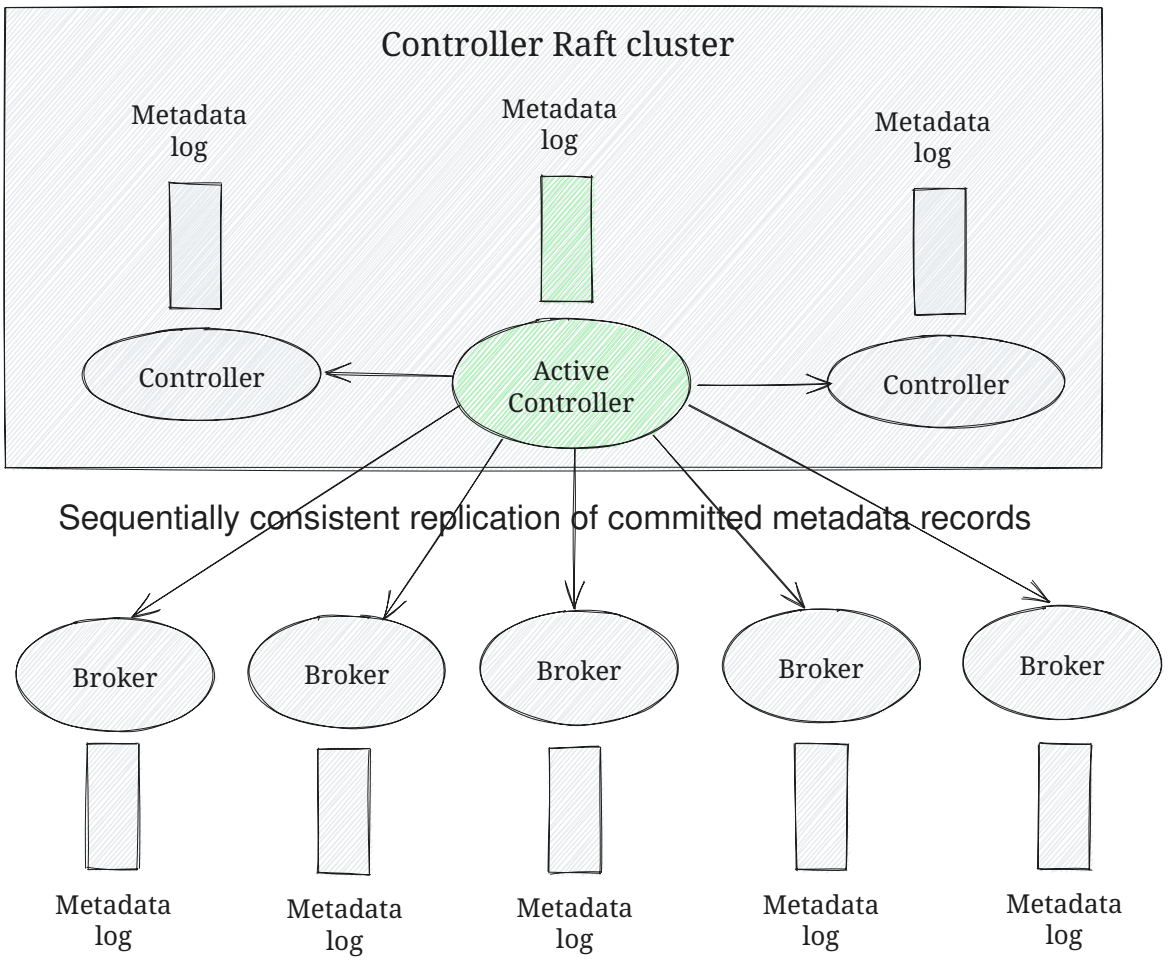

Fig 2. The metadata log and the controller and broker nodes.

Brokers and the replicas they host learn of changes via this metadata replication from controller to broker. The specifics of the metadata are covered in each relevant section.

1.2.2 Broker epochs and fencing basics

Broker epochs and broker fencing are fundamental concepts referred to in many parts of the replication algorithm and so are covered here briefly. These two concepts are covered in greater detail in later subsections.

A *broker epoch* denotes one uptime session of a broker. Each time a broker starts up it receives a broker epoch from the controller. This epoch is a cluster-wide monotonic integer. Unlike with the ZooKeeper based-controller, there is no ABA problem where a broker can be online, then offline but unobserved and then online again without the controller realizing. With KRaft, the broker registration process, which is carried out during broker start-up, involves the broker being assigned a monotonic broker epoch. This registration is added to the Raft metadata log which means this information survives controller fail-overs. The broker epoch plays a role in partition replication as an additional correctness barrier.

Brokers start in a fenced state and become unfenced once registered and having caught-up with the latest cluster metadata. Brokers send periodic heartbeats to the controller as a failure detection mechanism. Once the controller hasn't received a heartbeat from a broker for a specific time period, the controller fences the broker. A fenced broker loses any partition leadership it had and its partition replicas cannot be elected as leader either as long as the broker remains fenced. Once fenced, a broker can be unfenced by sending a heartbeat or registering again after having shutdown, rebooted or crashed. When a broker is unfenced via a heartbeat its broker epoch remains unchanged - the broker epoch denotes an uptime session.

The complete broker lifecycle from start-up, to steady-state, to shutdown is covered in the Broker Lifecycle section.

2.2 Polling-based data-plane replication

Kafka data-plane replication (partition data) is pull-based where leader replicas responds to fetch requests from followers.

2.2.1 Fetchers and fetch sessions

Each broker can be configured with one or more fetchers. Each broker-broker pair maintains a set of fetch sessions in each direction (based on the configured number of fetchers). Each fetcher is given responsibility for a subset of the partitions hosted on that broker.

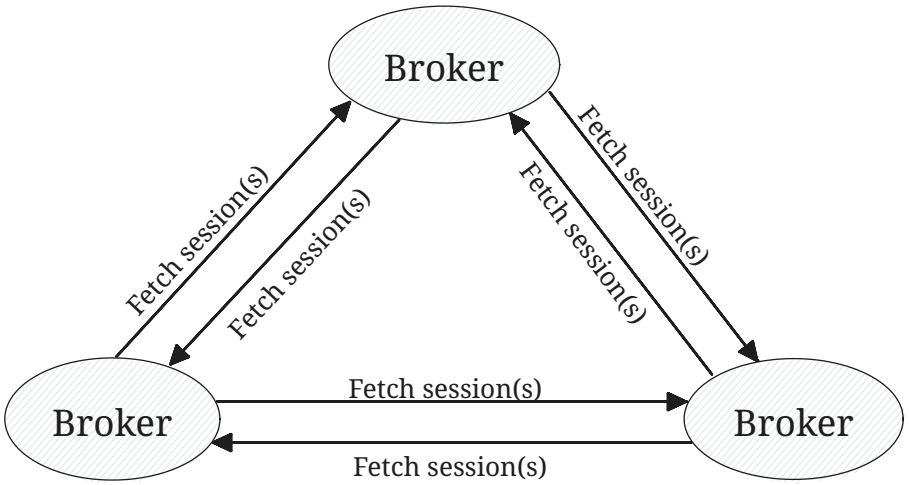

Fig 3. Each broker-broker pair maintains a number of fetch sessions in each direction. Each fetch request can include multiple partitions at a time.

Each session has a fetcher object on the source broker which fetches for all its partitions where the source broker hosts a follower and the destination broker hosts the leader.

For example, given three brokers [b1, b2, b3] with one fetcher, and three partitions [p0, p1, p2]:

- p0 -> Leader: b1, Followers: [b2, b3]
- p1 -> Leader: b2, Followers: [b1, b3]
- p2 -> Leader: b2, Followers: [b1, b3]

The following fetch sessions are maintained:

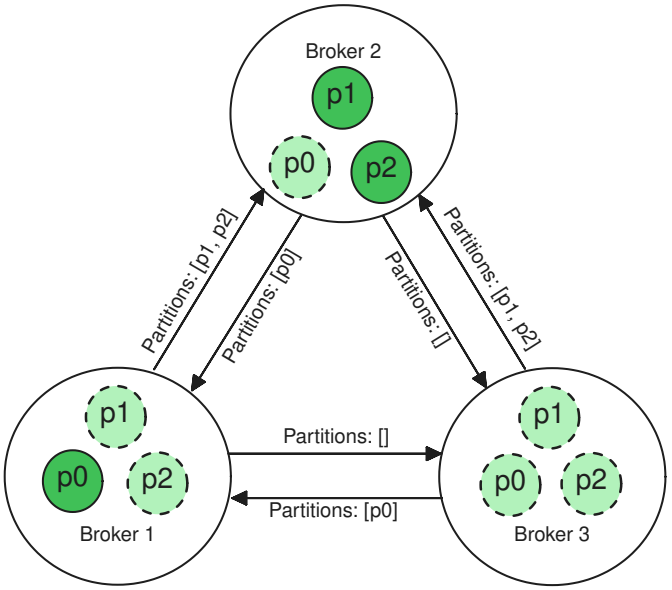

Fig 4. The partitions of each fetch session.

This means that physically speaking, polling is broker-to-broker not replica-to-replica but logically it is ok to think of fetch requests at the partition level for simplicity. When fetch requests are discussed in subsequent sections, know that fetch requests are broker-broker pair scoped and can contain the fetch state of multiple partitions in each fetch request.

Learn more about the underlying broker-to-broker fetch mechanism via KIP-227.

2.2.2 Partition replication

Note: Offsets start at 0.

Each replica stores the partition log itself as well as some additional fields:

- **Log End Offset (LEO)** which indicates the next offset of the log (or you can think of it as the exclusive end offset of the log). An empty log has an LEO of 0 as it is the next offset to be written.
- **High Watermark (HWM)** which marks the (exclusive) end of the committed log prefix. Like the LEO it is also exclusive so a value of 10 indicates that the log prefix 0-9 is considered committed and therefore stable. An empty log has a HWM of 0.
- **Log Start Offset** which indicates the lowest offset of the log. Limited data retention causes log garbage collection and this offset indicates the lowest offset that is still valid.
- **Last Stable Offset (LSO)** which denotes the (exclusive) end of the consumable log prefix and is related to Kafka's transaction system. This protocol description does not yet include transactions and is therefore focused on the HWM (when we omit transactions the LSO = HWM).

Each record (batch) has an associated offset and the leader epoch at the time it was written to the leader's log.

Partition records are replicated from the leader replica via fetch requests sent from each follower to the leader[1]. The follower indicates its position in the log via the **fetch offset** which is set to its LEO.

Other key fetch request fields are:

- The **broker epoch**, a cluster-wide monotonic integer that denotes one broker uptime session. Each time a broker restarts it gets a new broker epoch from the controller. Replicas hosted on a given broker use its broker epoch and sometimes this is referred to as the replica epoch. Leaders learn of broker epoch changes of followers via fetch requests (and not via metadata updates).
- The **current leader epoch** of the partition (that the follower knows of, meaning it could be stale). A leader epoch denotes one leader term and partition assignment.
- The **last fetched leader epoch** which is the leader epoch of the last fetched record

(required to detect log divergence).

If the request is valid the leader responds with a set of records starting from the fetch offset (if it has any).

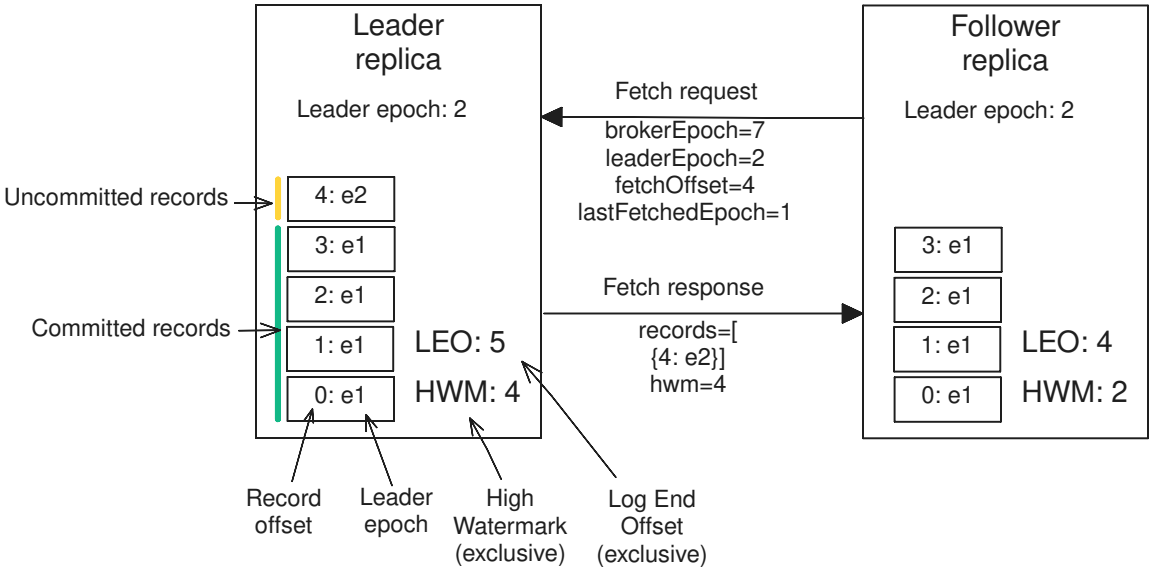

Fig 5. The partition state on each replica and a single fetch request/response cycle.

On the leader side, it stores state on each follower replica, including its LEO (known from the fetch offset of the last fetch request) and the broker epoch (also known from the last fetch request). The leader uses its knowledge of the LEO of its followers to advance the high watermark. The high watermark can only be advanced by the leader but is propagated to followers in the fetch responses. For this reason the high watermark is usually somewhat stale on the followers.

2.2.3 Adding and removing partitions to/from the fetchers

When a replica becomes a follower in a new leader epoch, it gets removed from the fetcher for the previous leader and added to the fetcher of the new leader, with a fetch offset equal to its LEO.

When a replica becomes a leader or gets removed from the replica set, the partition gets

removed from its current fetcher (if added).

2.3 Partition metadata

Each partition has certain metadata stored by the controller, each replica receives this metadata via metadata replication, though replicas do not know of some of the fields.

- **Replicas**: the members of the partition (the broker ids).
- **Leader**: the replica that is the current leader or NoLeader.
- **ISR (In-Sync-Replicas)**: The members which are currently “in-sync” (more on that next).
- **ELR (Eligible-Leader-Replicas)**: The members which were removed from the ISR but guaranteed to host the complete committed log and therefore are still leader candidates (more next). *Controller only.*
- **LastKnownELR**: A group of replicas which are consulted during unclean recovery. See Unclean recovery for more details. *Controller only.*
- **Leader epoch**: A monotonic integer. A leader epoch denotes a single leadership term and is incremented on each leader change and completed reassignment.
- **Partition epoch**: A monotonic integer. Each time the partition metadata is modified the partition epoch is incremented. This protects against concurrent or stale updates.
- **Adding**: A set of replicas that are in the process of being added as part of a reassignment.
- **Removing**: A set of replicas that are in the process of being removed as part of a reassignment.

The partition metadata does not include any information about offsets such as the HWM or LEO.

Note that the controller also stores the current broker epoch of each broker which also plays a role in partition metadata management. However, because this is broker-specific it is not stored in the partition metadata.

2.3.1 The leader epoch

Each election is assigned a monotonic leader epoch which is unique to that leader’s term of office. Additionally, reassignments (where the replicas that form the partition replica set is changed) also increment the leader epoch and so a single leadership term can span multiple epochs. Therefore one leader epoch is unique to one partition assignment of a given election term.

2.3.2 Metadata update deltas and materialization

Each change to the metadata of a partition is replicated as a separate change record delta. These deltas are replicated to each broker and then applied to the partition replicas in order to produce as full materialized metadata snapshots. Because deltas are replicated in batches, quick succession metadata changes can result in multiple deltas for the same partition being returned in the same batch. These deltas are merged in order with the current metadata to produce a new materialized snapshot of that partition's metadata and this is then acted on by the replica logic.

The protocol rules as described act on materialized metadata, not the deltas and some of the rules are necessary because the replicas are unable to see all intermediate states. For example, if a replica is quickly removed then readded to the partition replica set, then the replica logic may not see any change in the partition metadata (except if it were previously in the ISR but now is not).

2.4 The In-Sync-Replicas (ISR) and Eligible-Leader-Replicas (ELR)

The In-Sync-Replicas (ISR) and Eligible-Leader-Replicas (ELR) are both subsets of the partition replica set (note that subsets can equal the set). Which replicas are members of each is carefully curated by the controller and leader replica. These two sets lie at the heart of the replication protocol and this subsection discusses their place in the protocol and how they relate to similar concepts in other protocols such as Raft.

While the Kafka replication protocol is unique in many ways in the field of leader-follower replication algorithms, it shares the fundamental properties of:

- Leader Completeness
- Log Matching

Leader Completeness is the property that the leader hosts the complete committed log. Log Matching is the property that the stable prefix of the log is consistent across all replicas. These property names are commonly known from the Raft paper but are general properties for most, if not all, leader-based SMR protocols.

> Leader Completeness property: The leader hosts the complete committed log. More concretely, the leader log contains all offsets up to the True HWM.

> Log Matching property: The stable prefix of the log is identical on all replicas. More concretely, for all leader-follower pairs, the log prefix on both replicas is identical where the prefix is bounded by the min(Leader Local HWM, Follower Local HWM).

If a replica hosts the complete committed log, we'll refer to that as a **complete replica**. Likewise, an incomplete replica does not host the complete committed log.

2.4.1 Quorums

A common principle behind how these systems provide consistency under bounded failure scenarios is overlapping quorums. Systems prefer quorums rather than requiring all nodes to perform a given action as not all nodes will always be available. A quorum is basically some minimum subset of the nodes that must participate in an action for it to complete successfully and often they are majority based though this is not the case with Kafka.

All these leader-based systems have two types of quorum: a replication quorum and an election quorum.

In a majority quorum system such as Raft, we can define the quorums as:

- A replication quorum is some minimum subset of nodes that must receive a write for it to get committed.
- An election quorum is some minimum subset of nodes that have must all vote for the same node in the same election term for the election to complete successfully.

In majority-based systems such as Raft, the replication and election quorums are a majority quorums. If the Raft cluster has 3 nodes, then the first two nodes that confirm a write constitute the minimum majority quorum required to commit the entry. A Raft election quorum is the group of nodes that have voted for the same node in the same term and collectively form a majority.

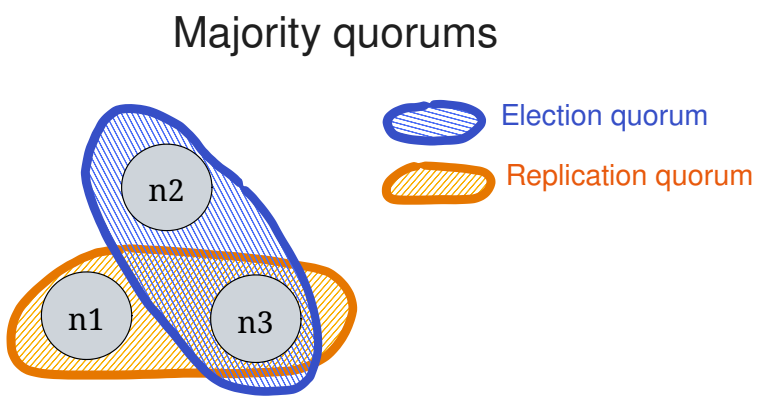

Fig 6. A Raft minimum replication quorum is guaranteed to overlap with a minimum election quorum.

Raft guarantees election and replication quorum overlap because quorums are majorities. For example, if the last committed entry exists on a majority quorum of [n1, n2] then any election quorum is guaranteed to intersect with [n1, n2]. For example, each of the possible majority quorums contains either n1 or n2: [n1, n2], [n2, n3], [n1, n3]. Raft chooses the node with the highest term and offset of the election quorum which means it always chooses a node with the complete committed log.

Quorums in Kafka are slightly different as they are actually explicit curated sub-groups which have concrete members at any one time. Whereas Raft quorums are implicit in how data is replicated and votes are cast, Kafka quorums are curated groups stored in metadata.

Therefore we can define the quorums in Kafkas as:

- A replication quorum is a concrete subset of nodes that must all confirm a write for it to get committed.
- An election quorum is a leader candidate pool where any member can be selected as the next leader.

The replication and election quorums are recorded in metadata and metadata can be modified by both the controller and the replica leader. The protocol is exercised by both the controller and the partition replicas and changes to this metadata must be disemminated.

The concrete sub-groups that form the replication and election quorums are:

- Replication quorum: The ISR known to the leader replica.
- Election quorum: The ISR known to the controller + the ELR (only known by the controller).

In Kafka, elections are performed by the controller which simply selects any replica that is a member of the ISR or ELR. In order to guarantee that the controller selects a replica with the complete committed log as leader, Kafka must guarantee that all replicas in the election quorum also exist in the replication quorum.

For example, if the leader-known-ISR is composed of [r1, r2] that means a record is only committed when it has been written to both r1 and r2. If a leader election were to occur at this moment, either r1 or r2 could be selected safely as all acknowledged messages MUST be hosted on both. As long as the controller-known-ISR + ELR consist of r1 and r2, everything lines up. This can be distilled into the following property:

> Leader Candidate Completeness: All replicas in the controller ISR and ELR host the complete committed log. More concretely, all such replicas contain all offsets up to the True HWM.

Given that there are no guarantees that the controller and replicas have the same knowledge of the quorums at the same time, extra safeguards are required to ensure this property holds. Additionally, there is some nuance to this property due to Kafka supporting asynchronous log writing which can result in local data loss on a replica. Therefore it is possible for a leader candidate to experience local data loss. However, this is detected and handled by the controller which removes such a replica from its candidate or leader status before a local data loss situation can propagate further. How this property is guaranteed, including handling of local data loss, is explained in sections Asynchronous storage and recovery and Replication correctness.

Violation of Leader Candidate Completeness can lead to violation of Leader Completeness after an election (resulting in data loss).

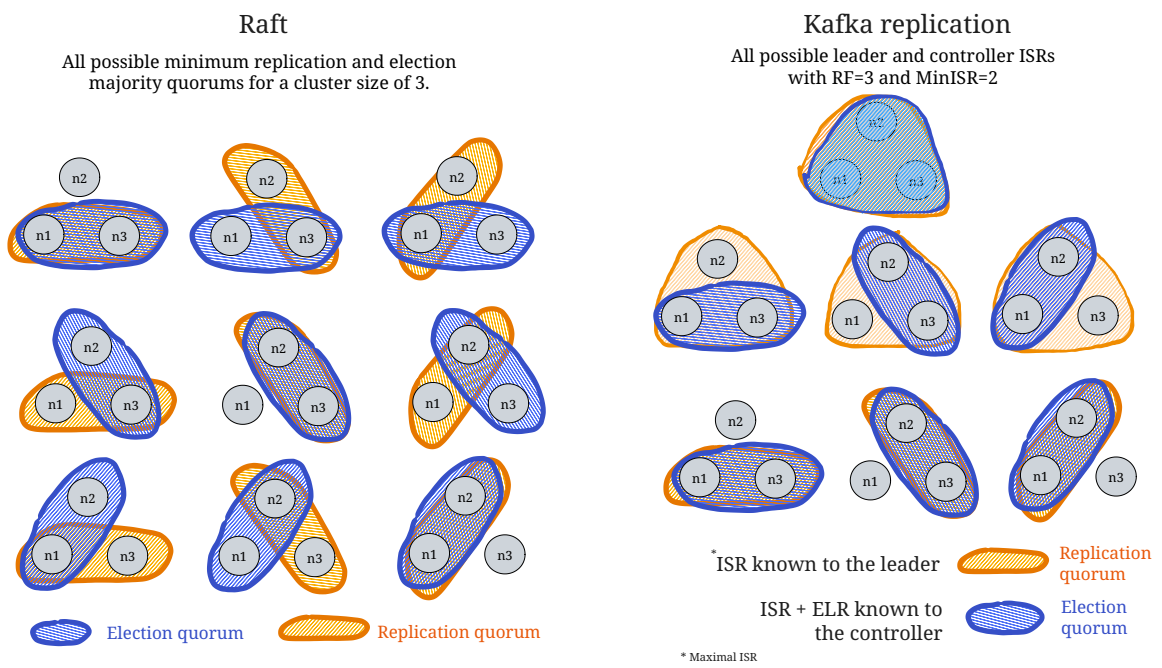

Fig 7. Raft quorums vs Kafka quorums. Note that Raft just needs an overlap of at least one node whereas Kafka requires that the election quorum overlap such that it is a subset of the replication quorum.

Note that the leader ISR shown in the above figure refers to the leader's Maximal ISR (see The Maximal ISR).

2.4.2 ISR and ELR membership

The ISR shrinks when the partition is experiencing degraded conditions, such as brokers going offline, getting network partitioned or falling behind due to degraded hardware or an imbalanced stressful workload. The ISR can shrink to empty in the most extreme cases. The election quorum on the other hand should not shrink too far, as once it is empty, clean elections are not possible. Both availability and durability are enhanced by maintaining as large an election quorum as is safely possible - this is the purpose of the ELR.

The HWM cannot advance when the leader-known-ISR (replication quorum) is smaller than the MinISR (the configured minimum ISR size for HWM advancement). The insight is that when the HWM cannot advance, ISR replicas which get removed from the ISR will remain complete (host the complete committed log) despite no longer being a member of the replication quorum. The controller can essentially freeze the election quorum when the replication quorum is too small for the HWM to advance.

Therefore when the controller removes a replica from its ISR it can be moved to the ELR when ISR < MinISR. When the ISR grows large enough for the HWM to advance again, the controller empties the ELR as any members could no longer be guaranteed to host all committed entries.

When a partition has no leader it means that the ISR has shrunk to empty and either the ELR is empty or it is non-empty but all members are currently fenced. In the latter case, as soon as a fenced ELR member becomes unfenced, it can be elected as leader. In the former case, when the ELR is empty, no clean election is possible and the controller will resort to unclean recovery.

ISR membership is mostly managed by the leader replica which can add and remove members. However, if the ISR becomes empty, then an ELR member can be moved from the ELR to the ISR (to become the new leader) by the controller.

ELR membership is entirely managed by the controller. The ELR is discussed in further detail in Asynchronous storage and recovery.

2.4.3 ELR not “ISR with strict MinISR semantics”

An alternative to the ELR would be to prevent the ISR from shrinking below MinISR. However, due to the long history of the ISR and its role in tooling, there is an expectation that the ISR only contains in-sync and unfenced replicas. If the ISR could also include fenced replicas (because the broker died) then this could make reasoning about the state of a partition harder. Therefore, the ELR exists so that we benefit from essentially a strict MinISR that avoids the Last Replica Standing issue, but maintain the meaning of the current ISR.

2.4.4 Safety and liveness guarantees

Majority quorum systems provide safety and liveness guarantees based on the number of

nodes, where the size is determined by the formula: `Cluster size = 2f + 1`. Here `f` is the threshold of node failures we want the system to tolerate.

However, Kafka is not majority-quorum based and so does not use this formula. Instead safety guarantees are based on a tunable configuration called `min.insync.replicas` which we'll refer to as the **MinISR**.

The safety and liveness formula for Kafka is: `MinISR = f + 1` or alternatively, `f = MinISR - 1`.

Clusters can be sized more freely with the number of replicas (Replication Factor or RF) and the MinISR being somewhat decoupled.

For example, with RF=5, an administrator could:

- choose a simple majority of MinISR=3 which balances safety and availability.
- go for higher durability with MinISR=4.
- go for higher availability with MinISR=2.

2.5 Produce request durability level (acks)

Producers can indicate the level of durability via the producer config `acks`.

This config has three levels:

- `-1` or `all` tells the leader replica that it can only send a positive acknowledgment to the producer when all replicas in the ISR have confirmed they have written the request records.
- `1` tells the leader it can send a positive acknowledgement as soon as it has written the request records to local storage.
- `0` is basically fire-and-forget.

The `acks` producer config only pertains to when the leader acknowledges a produce request - it does not affect replication itself. As far as replication and the HWM is concerned, it's all about the records written to the leader log and all records on disk are treated equally.

The main focus of this protocol description is `acks=all` however, where relevant, `acks=1` will also be discussed.

2.6 min.insync.replicas (MinISR)

When a producer sends a produce request with acks=all, it is logical to expect that the records will be stored redundantly by the time a positive acknowledgement is received. However, if the

ISR has shrunk down to the leader, then an acknowledgment can be sent despite the records having only been written to the leader log.

To prevent this, the topic config `min.insync.replicas` exists, referred to from now on as the **MinISR**.

The MinISR determines the number of replica failures a partition can tolerate without loss of data or availability. Where `f` is the number of failures to be tolerated, the MinISR formula is: `MinISR = f + 1`.

The MinISR does not place a lower-bound size on the ISR itself but certain actions are limited to when the ISR >= MinISR, such as:

1. HWM advancement.
2. Acceptance of `acks=all` produce requests.
3. Completion of partition reassignments.

When the ISR size shrinks below MinISR, the following things happen:

- The HWM cannot advance.
- Any reassignment of that partition cannot complete.
- All new `acks=all` produce requests are immediately negatively acknowledged (with the `NotEnoughReplicas` error) until the ISR expands to >= MinISR again.
- Any pending uncommitted `acks=all` requests already written to the log are negatively acknowledged with the error `NotEnouhReplicasAfterAppend`.

Regarding produce requests, this fail-fast mechanism helps slow replicas catch-up quickly as the leader log stops advancing. On the producer side, if the producer is using idempotency, it will simply retry on these errors and the broker will avoid duplicates from being added to the partition log. Therefore from the application side, the code does not need to handle these errors explicitly.

Note, producer idempotency is currently out-of-scope in this protocol description.

2.7 The high watermark (HWM)

The high watermark marks the end of the committed and stable log prefix from the uncommitted and unstable log suffix.

When transactions are not used the Last Stable Offset (LSO) and the High Watermark (HWM) point to the same offset. The LSO is generally omitted from this protocol description as transactions are not included (for now).

Only the stable portion of the log is visible to consumers.

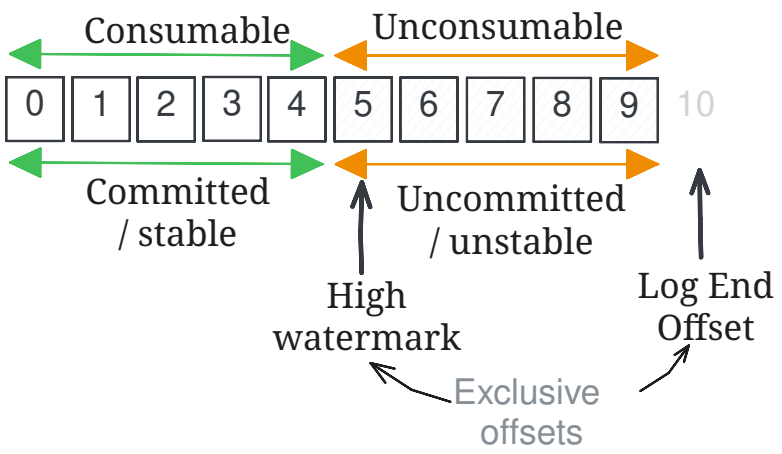

Fig 8. The high watermark separates the committed/stable and consumer visible prefix from the uncommitted/unstable suffix.

The leader advances the high watermark based on the current fetch offsets of its followers and the current Maximal ISR. The Maximal ISR is a variant of the ISR stored by the leader replica only. It is necessary when the leader performs ISR changes. The Maximal ISR is essentially the union of the current ISR and a proposed new ISR that has not yet been committed. See The Maximal ISR in the Replication Correctness section for more details.

The leader maintains state on each follower, including its broker epoch (sometimes referred to as the replica epoch), the leader epoch and its fetch offset. The HWM can advance when the `ISR >= MinISR` and all followers in the Maximal ISR have a fetch offset higher than the current HWM (plus some additional rules covered later). When the HWN advances, any pending produce requests with records lower than the new HWM can be positively acknowledged.

The HWM advancement can be triggered on:

- A change to the known fetch offsets of the followers (when a fetch request is received).
- When the ISR shrinks (as after shrinkage, all members of the ISR may have a high enough fetch offset for the HWM to advance).
- When the ISR expands (as it may cause the ISR size to reach the MinISR).
- When the leader receives a produce request (if the leader is the sole member of the ISR and MinISR=1).

Kafka provides linearizable semantics to clients which means consumers should be able to consume acknowledged records immediately after a write completes, in the same real-time order they were written. This same order should not change, for consumers that are tailing the log as well as consumers which consume the log much later. Therefore consumers are only allowed to read up to the HWM as reading from the unstable suffix of the log could result in reads which are not consistent with the acknowledged write history. HWM advancement

triggers both the producer acknowledgement and consumer visibility of the records.

Note however that it is possible for records to be read by consumers that no producer received a positive acknowledgment for. These cases include:

- The producer, broker or network connection failed before the positive acknowledgement could be sent/received.
- An acks=all request gets written to the leader log but then before the records get committed the ISR size drops lower than MinISR which results in a negative acknowledgement sent to the producer. However, once written to the log, the request records will eventually get replicated (as long as a leader election doesn't occur before that can happen). This means these records can get committed (despite having been negatively acknowledged) and read by consumers. This is where idempotent writes are important as when employed by producers, as soon as the ISR recovers to reach MinISR replicas, the producer will receive a positive acknowledgement for those records (without duplication).

2.8 Leader elections

Leadership needs to change according to the conditions and we'll study how those changes are performed in this section. The leader, ISR and ELR are durably and consistently stored in the partition metadata by the KRaft controller.

Only the controller can elect leaders. A leader election occurs when any of the following occur:

- The current leader gets fenced.
- There is currently no leader but a replica becomes a leader candidate.

When a broker gets fenced due to the lack of a heartbeat or because it has shutdown, the controller performs the following (atomically) to each partition for which it hosted the leader replica:

- Remove the replica as leader.
- Remove the replica from the ISR.
- Choose a new leader:
    - If there is at least one replica in the ISR (that is not shutting down), select one as the new leader.
    - Else if there is one replica in the ELR which is unfenced then select one as the new leader and move it from the ELR to the ISR.
    - Else set the leader as NoLeader.
- If the new ISR is smaller than the MinISR, then add the fenced former leader to the ELR.

- Increment the leader epoch.
- Increment the partition epoch.
- Commit the new partition metadata.

Fenced replicas get added to the ELR when the new ISR is too small for the HWM to advance. Because the HWM cannot advance, the fenced replica will still continue to host the complete committed log (and is therefore still a leader candidate). When a partition has no leader it means that the ISR has shrunk to empty and either the ELR is empty or it is non-empty but all members are currently fenced. In the latter case, as soon as a fenced ELR member becomes unfenced, it can be elected as leader. In the former case, when the ELR is empty, no clean election is possible.

The partition replicas will learn of any change to the partition metadata via PartitionChange records in the metadata log that is replicated from the controller to the brokers.

2.9 Modifying the ISR (and ELR)

Both the controller and the partition leader can modify the ISR and because both act independently, the partition epoch is required to guarantee consistency. The ELR is only modified by the controller and based solely on the changes made to the ISR. The replica leader has no knowledge of the ELR at all.

The controller is able to shrink the ISR down to empty but is not allowed to expand it (except for moving an ELR member to the ISR when the ISR is empty). It is the leader's job to expand the ISR as expansion is based on knowledge of the follower log and the controller knows nothing of this. The leader adds a follower to the ISR when the out-of-sync follower has become "in-sync".

Shrinking the ISR is a safe operation in of itself. It can affect availability, but the MinISR places a lower bound on the number of complete replicas (bar broker failures). If shrinking the ISR makes it smaller than MinISR then the removed replica(s) get moved to the ELR such that the combined ISR + ELR size >= MinISR.

Shrinkage can be needed to maintain availability and bound latency. If a member of the ISR goes offline or falls behind, keeping it in the ISR does not affect consistency, but it can prevent or slow down the advancement of the HWM which affects producer acknowledgement and consumer visibility of records. For example, when the ISR=[r1,r2,r3] but r3 is offline, then it will block HWM advancement and a shrinkage down to [r1,r2] will enable the HWM to advance again.

Without extra controls, expanding the ISR would be a potentially unsafe operation as getting it wrong could violate the Leader Candidate Completeness property, leading to data loss. It is the leader that expands the ISR and it does so based on some rules. The rules for qualifying a

follower as in-sync with the leader can be categorzied as those pertaining to latency/availability and those pertaining to consistency.

Latency/availability rules:

- The follower log is either not lagging behind the leader log, or is only lagging a very small amount.
- The leader has received a fetch request within a short time frame (10 seconds by default).

Consistency rules:

- The follower is already a member of the ISR.
- The follower is not currently a member of the ISR but:
  - The out-of-sync follower fetch offset has reached the HWM.
  - The out-of-sync follower fetch offset has reached the Leader Epoch Start Offset (LESO).
  - The leader has received a fetch request from the out-of-sync follower in the current leader epoch.

Shrinking the ISR is based on the latency/availability rules. Expanding the ISR is based on all rules.

There is further discussion of the above rules and high watermark advancement in Replication Correctness.

2.9.1 Controller initiated changes

The controller can initiate the following ISR/ELR changes:

1. Shrink the ISR when a member becomes fenced. If the resulting ISR is below MinISR, then the fenced replica is added to the ELR.
2. Move an ELR member to the ISR when the ISR is empty (electing the former ELR member as leader).

Replicas learn of these changes via the metadata log.

2.9.2 Leader initiated changes

A leader replica can both expand and shrink the ISR due to changes in conditions. The leader replica sends an AlterPartition request to the controller with the proposed ISR. The leader does not wait for a reply, it immediately updates its Maximal ISR to be the union of the current ISR and the new proposed ISR. Later it will commit or rollback the change depending on the response from the controller. HWM advancement is always based on the Maximal ISR.

The AlterPartition request contains the following key data:

- Broker epoch of the leader replica
- Current partition epoch
- Current leader epoch
- The leader replica
- The proposed ISR
- The broker epoch of any added replicas.

The controller will accept an AlterPartition request only when the following is true:

- The broker epoch of the leader is correct.
- The partition epoch matches the current metadata.
- The broker epoch of any added replicas is correct.

If the leader receives a success response it commits the change to its local metadata:

- Sets the ISR and maximal ISR to the accepted ISR.
- Updates the partition epoch.

If the leader receives an error response, it rolls back to the prior committed metadata (resetting the maximal ISR to the ISR).

Lastly the leader may advance the HWM if the updated maximal ISR change now allows the HWM to advance.

Note that it is possible for the leader to receive the metadata record for the partition change before receiving the AlterPartition response, therefore the local partition metadata should only be updated if the partition epoch of the update/response is higher than the currently known partition epoch.

Finally, the controller may modify the ELR based on the ISR changes:

- If the ISR shrinks it may add the removed replica(s) to the ELR.
- If the ISR expands and has reached the MinISR then the ELR is emptied.

Partition changes are described in further detail, including the reasons behind the rules explained in this subsection, in Replication correctness.

2.10 Partition changes and replica transitions

Fetching is impacted by leader epoch changes in the following ways:

- Inflight fetch requests may get rejected due to the recipient no longer being the leader or because the leader epoch in the fetch request does not match the leader epoch of the recipient. The recipient may not be the leader because of an election or because it got removed from the replica set (both of which cause the leader epoch to increment). This mismtach can occur because the recipient learned of the leader epoch change first, or because the sender learned first. Only once both sides have learned of the change can fetching be successful.
- The local partition may need to either/and:
  - be removed from the current fetcher.
  - added to the fetcher of the new leader.

When a replica learns of a partition change, it is either instructed to be a leader or follower. This role can match its current role or could be a role change. The action taken by the replica depends on the type of role change and whether the leader epoch has changed.

Remember that due to metadata delta materialization, the role change could span multiple metadata changes which are hidden from the replica logic itself.

Many of the actions taken in these transitions are discussed in Replication Correctness.

2.10.1 Transition from follower to leader

- The partition is removed from its current fetcher.
- The replica ensures all follower state is cleared.
- The replica sets the Leader Epoch Start Offset (LESO).
- If the ISR is only the leader and MinISR=1 then the leader can advance the HWM to its LEO.

2.10.2 Transition from leader-to-leader in the same leader epoch

If the partition change is the result of an ISR shrink or ISR expansion, this could be the change required to allow the HWM to advance so the leader checks if it can advance the HWM.

2.10.3 Transition from leader-to-leader in a new leader epoch

The leader epoch change could be due to an election or a reassignment. If caused by an election, then between the leader's current knowledge of the partition metadata and the new update, it must have lost leadership and then been reelected. Remember that replicas react to materialized snapshots of potentially multiple partition change deltas compressed into one.

The leader must:

- Clear the last fetch time of out-of-sync followers. This ensures that these followers cannot be added to the ISR until a fetch is received in this leader epoch.
- The replica sets the Leader Epoch Start Offset (LESO).

2.10.4 Transition from leader-to-follower

The partition fetch state is added to the fetcher whose destination broker is the broker that hosts the new leader. The initial fetch state includes the fetch offset (the current LEO), the broker epoch and the new leader epoch.

2.10.5 Transition from follower-to-follower in the same leader epoch

The follower does not need to do anything as no leader change has occurred. ISR changes are immaterial to the follower.

2.10.6 Transition from follower-to-follower in a new leader epoch

The partition is removed from the current fetcher and added to the fetcher where the destination broker is the new leader (which may be the same leader as before). The initial fetch state includes the fetch offset (the current LEO), the broker epoch and the new leader epoch.

Footnotes

1. Remember fetch requests are broker-to-broker requests which include multiple partitions, but for simplicity we'll often talk about fetch requests with regard to a particular partition.

3. Detecting and avoiding log divergence

Distributed replicated log designs such as Raft, Apache Kafka and Apache BookKeeper share the common property that the stable portion of the log on each node must match each other. This is known as the Log Matching property. With Kafka this is expressed as the following:

> Log Matching property: For all leader-follower pairs, the log prefix on both replicas is identical where the prefix is bounded by the min(Leader Local HWM, Follower Local HWM).

The committed prefix of the replicated log must be consistent across all nodes. The committed portion of the log is considered stable and once stable any divergence will remain in place.

Therefore log matching is enforced in the unstable suffix via log truncation on followers. When a leader is elected, it treats its entire log as the one-source-of-truth and all followers must ensure that their logs are consistent with the leader. Post-election, if a follower has some log divergence from the leader, it must truncate the divergent portion of its log before replication between the leader and follower can make progress. In this section we'll discuss how logs can diverge and the truncation mechanism.

3.1 How logs can diverge

After a leader election, the log of the leader may have diverged from the logs of the followers.

Remember that offsets start at 0 and the LEO is the "last offset in the log + 1", so when the log is empty the LEO=0 and the same applies to the HWM.

Here is an example of how a follower log diverges from the leader log after an election.

1. Partition state: `Replicas [r1, r2, r3], ISR=[r1, r2, r3], Leader=r1` .
   - Replica 1 (leader): `LEO=0, HWM=0, LeaderEpoch=1, Log=[]`
   - Replica 2: `LEO=0, HWM=0, LeaderEpoch=1, Log=[]`
   - Replica 3: `LEO=0, HWM=0, LeaderEpoch=1, Log=[]`
2. Leader receives a write with record A in leader epoch 1. Replicas r2 and r3 send a fetch request with a fetch offset of 0 and the leader sends a response with record A and a HWM of 0.
   - Replica 1: `LEO=1, HWM=0, LeaderEpoch=1, Log=[{data=A,le=1}]`
   - Replica 2: `LEO=1, HWM=0, LeaderEpoch=1, Log=[{data=A,le=1}]`
   - Replica 3: `LEO=1, HWM=0, LeaderEpoch=1, Log=[{data=A,le=1}]`
3. Replicas r2 and r3 send another fetch request with fetch offset 1. The leader knows both itself and replicas r2 and r3 have offset 0. This means that all replicas in the ISR have confirmed the write and so the leader advances the HWM. The leader also receives a write with record B in leader epoch 1.
   - Replica 1: `LEO=2, HWM=1, LeaderEpoch=1, Log=[{data=A,le=1},{data=B,le=1}]`
   - Replica 2: `LEO=1, HWM=0, LeaderEpoch=1, Log=[{data=A,le=1}]`
   - Replica 3: `LEO=1, HWM=0, LeaderEpoch=1, Log=[{data=A,le=1}]`
4. Before record B can be replicated to the followers, the controller fences replica 1 and elects replica 2 as the leader in leader epoch 2.
   - Replica 1: `LEO=2, HWM=2, LeaderEpoch=1, Log=[{data=A,le=1},{data=B,le=1}]`
   - Replica 2 (new leader): `LEO=1, HWM=0, LeaderEpoch=2, Log=[{data=A,le=1}]`
   - Replica 3: `LEO=1, HWM=0, LeaderEpoch=1, Log=[{data=A,le=1}]`
5. Replica 1 is then told to become a follower in leader epoch 2. The new leader, r2, now

has a divergent log from follower replica 1 (it has fewer records than the follower). Replica 1 must remove offset 1 from its log else its log will continue to diverge from that of the leader.

3.2 The truncation mechanism

There have been at least two truncation mechanisms prior to the current truncate-on-fetch design though only the current approach is documented here.

Log divergence is detected by comparing the requested offset and the last leader epoch fetched against the state of the leader log. Each log record has an offset and a leader epoch (the leader epoch at the time it was written to the leader log).

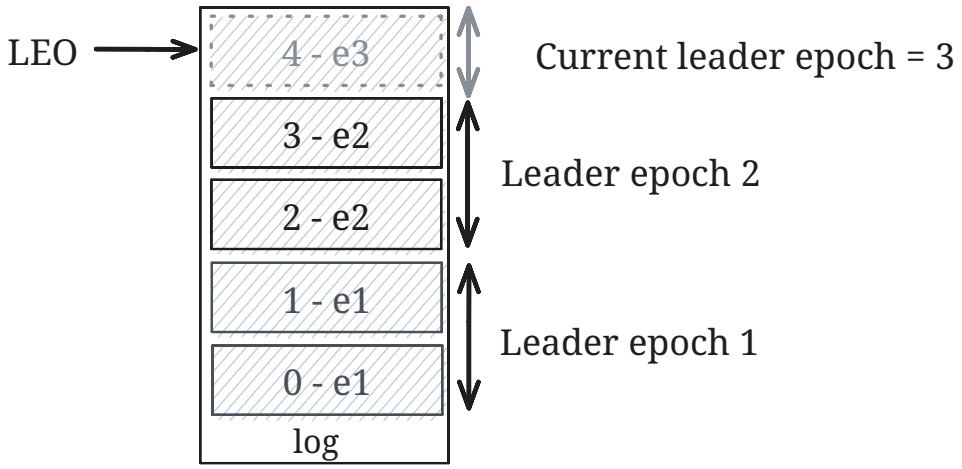

Fig 1. The log has four records, with two written in leader epoch 1 and two written in leader epoch 2. No records have yet been written in the current leader epoch 3 so the start offset of epoch 3 is the LEO.

Log divergence is detected by the leader when a follower sends its first fetch request after an election. The fetch request includes the fields `fetchOffset` which aligns with its LEO, and `lastFetchedEpoch` which is the leader epoch associated with its last log record.

Upon receiving the fetch request, the leader performs an *End Offset for Leader Epoch* lookup against a local cache which returns an epoch and offset such that:

- The epoch is the highest leader epoch in its log <= target epoch.
- The offset is the exclusive end offset of that leader epoch.

If there is no such leader epoch then the lookup returns:

- The requested (last fetched) epoch.
- The start offset of the lowest epoch in the log as this will act as the exclusive end offset to

be truncated.

Either of the following results indicate a diverged log:

- The epoch of the End Offset for Leader Epoch lookup is less than the lastFetchedEpoch of the fetch request.
- The offset of the End Offset for Leader Epoch lookup is less than the fetchOffset of the fetch request.

When a diverged log is detected, the leader sets the field `divergedEpoch` in its fetch response, with the value of the lookup epoch and end offset.

The follower, on seeing the divergedEpoch field being non-null, performs its own End Offset for Leader Epoch lookup, using the epoch in the divergedEpoch field. It then takes the minimum of the offset of its own lookup and the offset in the divergedEpoch field. It then truncates its log to that offset (exclusive).

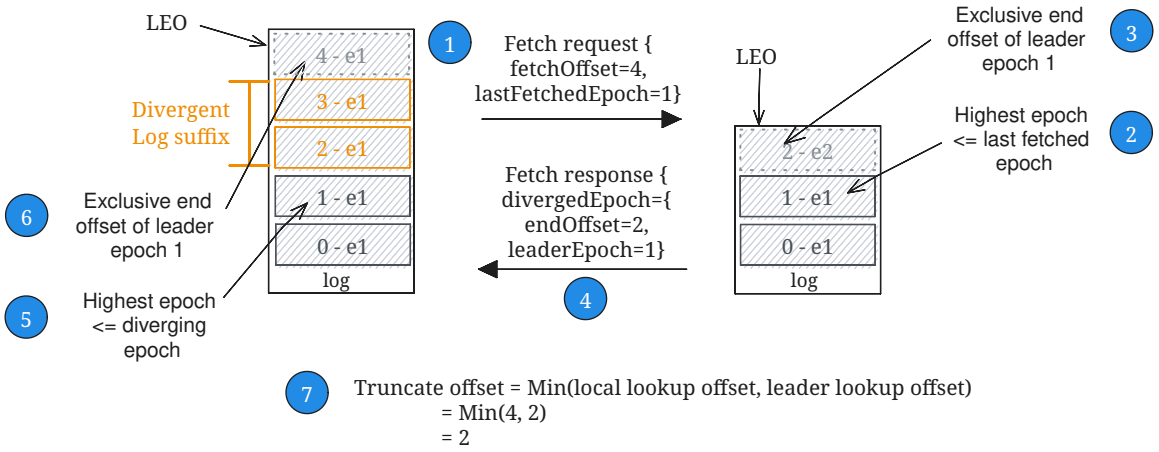

Fig 2. An example of the truncation mechanism at work.

Each replica stores a Start Offset for Leader Epoch cache to perform these lookups rather than having to read the log itself.

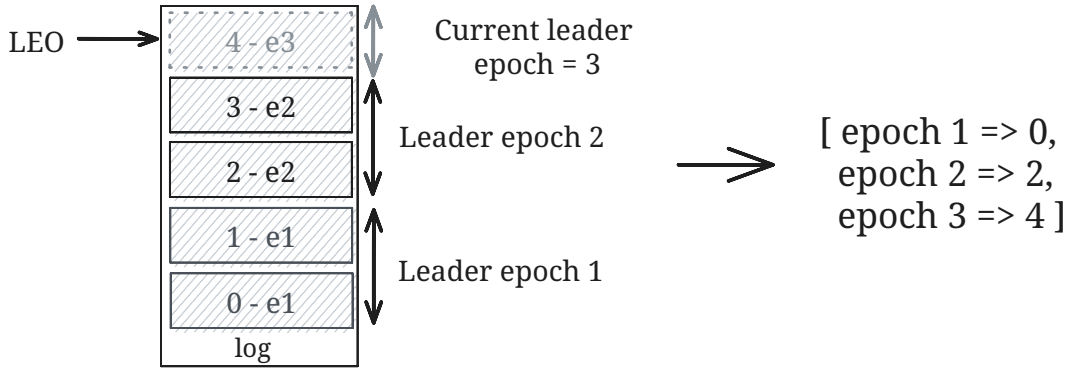

Fig 3. A leader epoch start offset cache speeds up lookups.

This can be difficult to reason about, so the following section goes through multiple possible types of divergence and how the truncation mechanism works.

3.3 Truncation with different divergence examples

The following examples show the immediate aftermath of a leader election where both leader and follower have learned of the new leader epoch. Both follower and leader have log records which were written in prior leader epochs. The examples show how the first fetch request and response result in either no truncation of the follower log (due to no divergence) or some truncation when divergence has occurred.

Each example shows the following replica state:

- Leader epoch. The current leader epoch the replicas know of.
- Latest epoch. This is the last leader epoch stored in the log itself.
- Cache. This is a map of Start Offset for Leader Epoch which can be built by reading the log. It is used to calculate the End Offset for Leader Epoch for the divergingEpoch field in the fetch response.

3.3.1 Case A (no divergence). Follower log behind the leader.

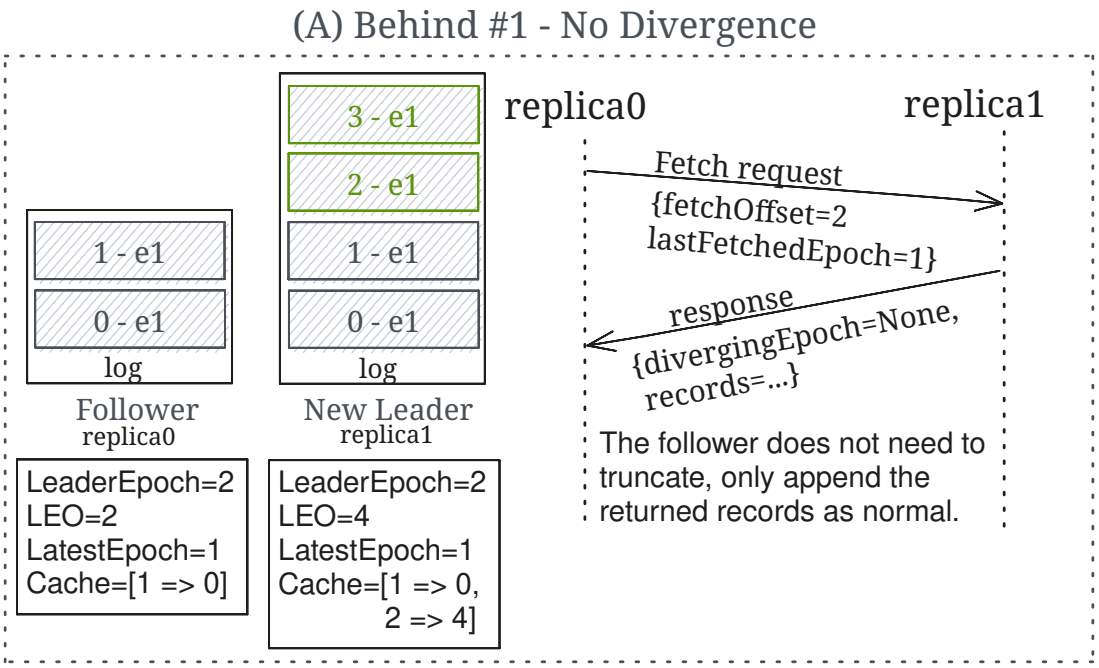

Fig 4. The leader log has additional records of the same leader epoch which means there is no log divergence. The follower simply needs to append the returned records as normal.

3.3.2 Case B (no divergence). Follower log behind the leader

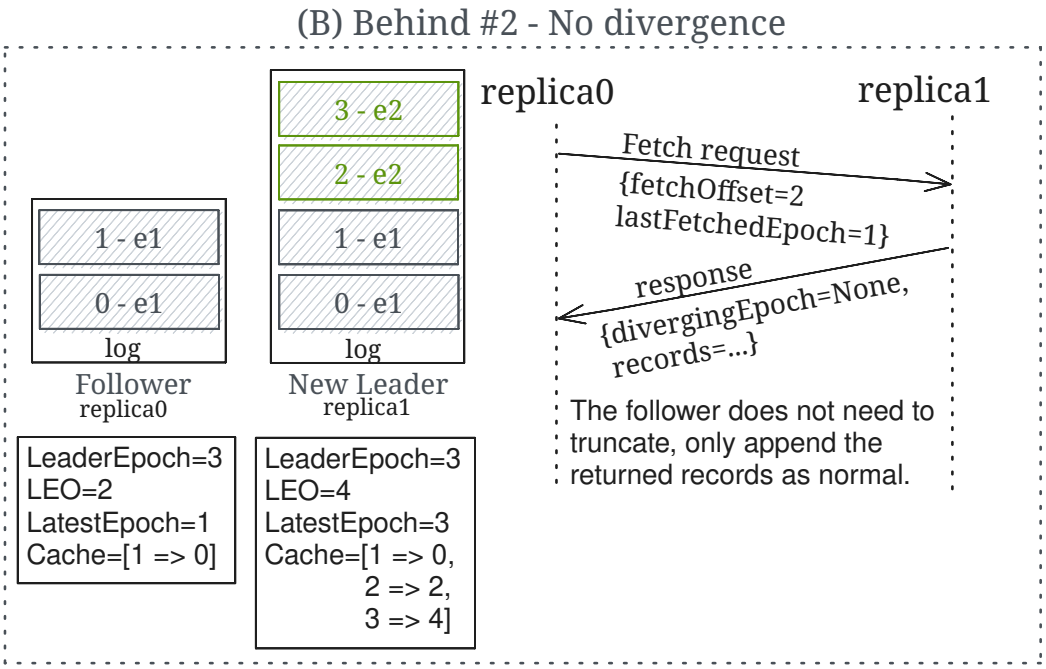

Fig 5. The leader log has additional records with a higher leader epoch which means there is no log divergence. The follower simply needs to append the returned records as normal.

3.3.3 Case C (divergence). Follower log ahead of the leader

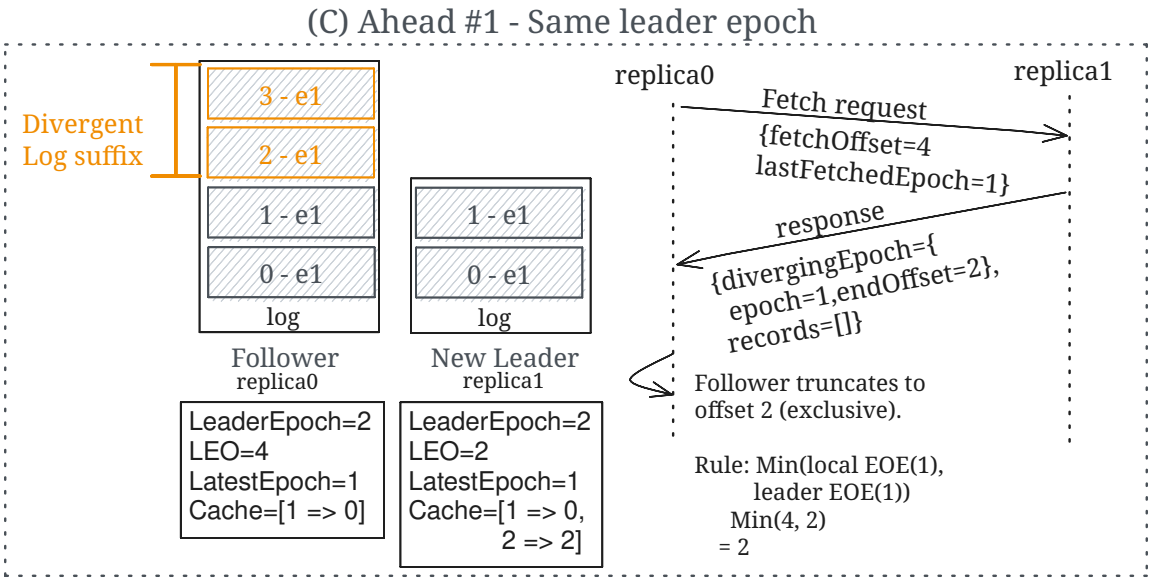

Fig 6. The follower log is ahead of the new leader log, with extra records of leader epoch 1.

While the lastFetchEpoch matches the latest epoch on the leader, it detects that the fetch offset is too high. Therefore it responds with a diverging epoch of {epoch=1, endOffset=2}. The follower selects the minimum of:

- It's local End Offset for Epoch (EOE) for leader epoch 1 = offset 4.
- The leader's End Offset for Epoch for leader epoch 1 = offset 2

Therefore it selects truncation offset (exclusive) = 2.

3.3.4 Case D (divergence). Follower log ahead of the leader

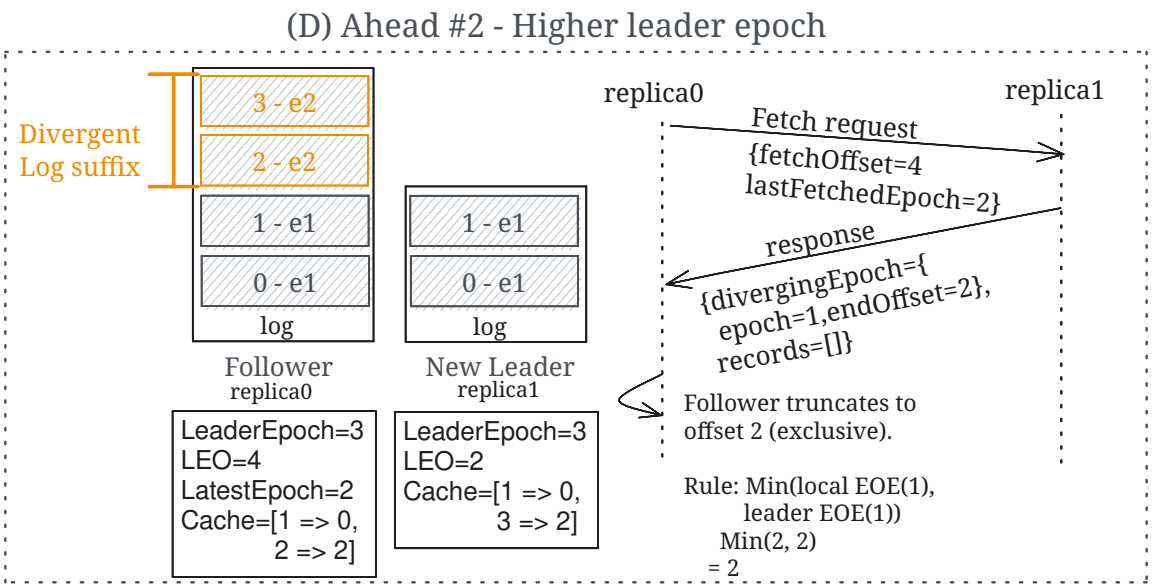

Fig 7. The follower log is ahead of the new leader log, with extra records with a higher leader epoch.

The lastFetchEpoch does not exist in the leaders Start Epoch for Leader Epoch cache. It returns a diverging epoch for the requested epoch (epoch 1) and the start offset of the next highest epoch (the current leader epoch which is 3) as the endOffset. The start offset of epoch 3 is the LEO.

The follower selects the minimum of:

- It's local End Offset for Epoch (EOE) for leader epoch 1 = offset 2.
- The leader's End Offset for Epoch for leader epoch 1 = offset 2

Truncation offset (exclusive) = 2.

3.3.5 Case E (divergence). Follower log forks from the leader log

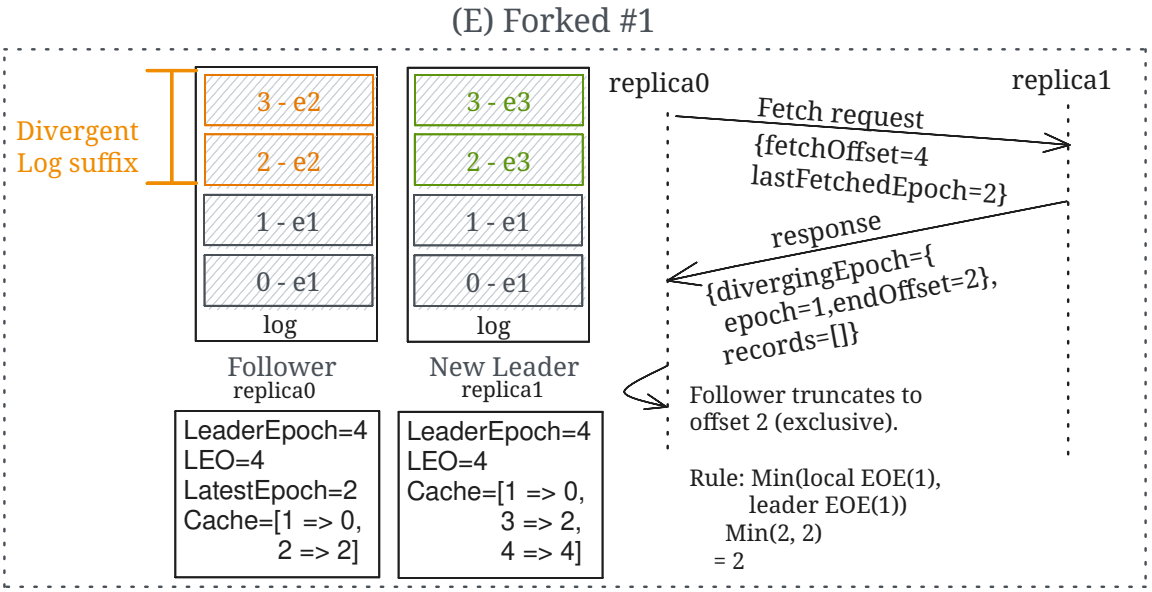

Fig 8. The follower log forks from the new leader log. Each fork has a different leader epoch.

The lastFetchEpoch does not exist in the leaders Start Epoch for Leader Epoch cache. It returns a diverging epoch with the highest epoch <= requested epoch (which is epoch 1) and the end offset as the start offset of the next highest epoch (which is offset 2).

The follower selects the minimum of:

- It's local End Offset for Epoch (EOE) for leader epoch 1 = offset 2.
- The leader's End Offset for Epoch for leader epoch 1 = offset 2

Truncation offset (exclusive) = 2.

3.3.6 Case F (divergence). Follower log forks from the leader log

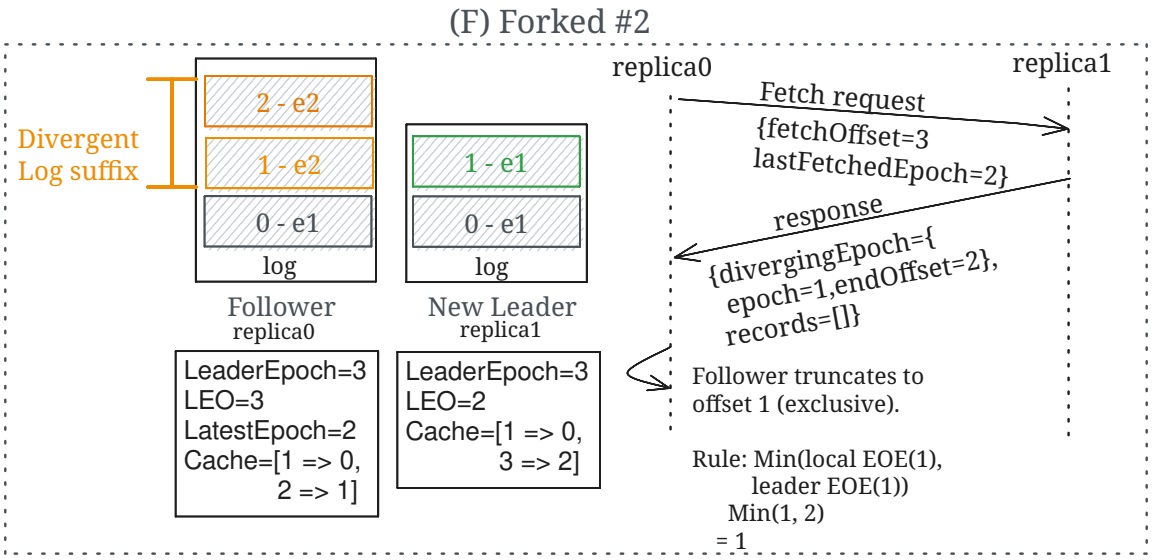

Fig 9. The follower log forks from the new leader log. The follower fork has a different leader epoch but the leader fork simply has extra records of epoch 1.

The lastFetchEpoch does not exist in the leaders Start Epoch for Leader Epoch cache. It returns a diverging epoch with the highest epoch <= request epoch (epoch 1) and the end offset as the start offset of the next highest epoch which in this case is the current leader epoch (offset 2).

The follower selects the minimum of:

- It's local End Offset for Epoch (EOE) for leader epoch 1 = offset 1.
- The leader's End Offset for Epoch for leader epoch 1 = offset 2

Truncation offset (exclusive) = 1.

3.3.7 Case G (divergence). Follower log forks from the leader log

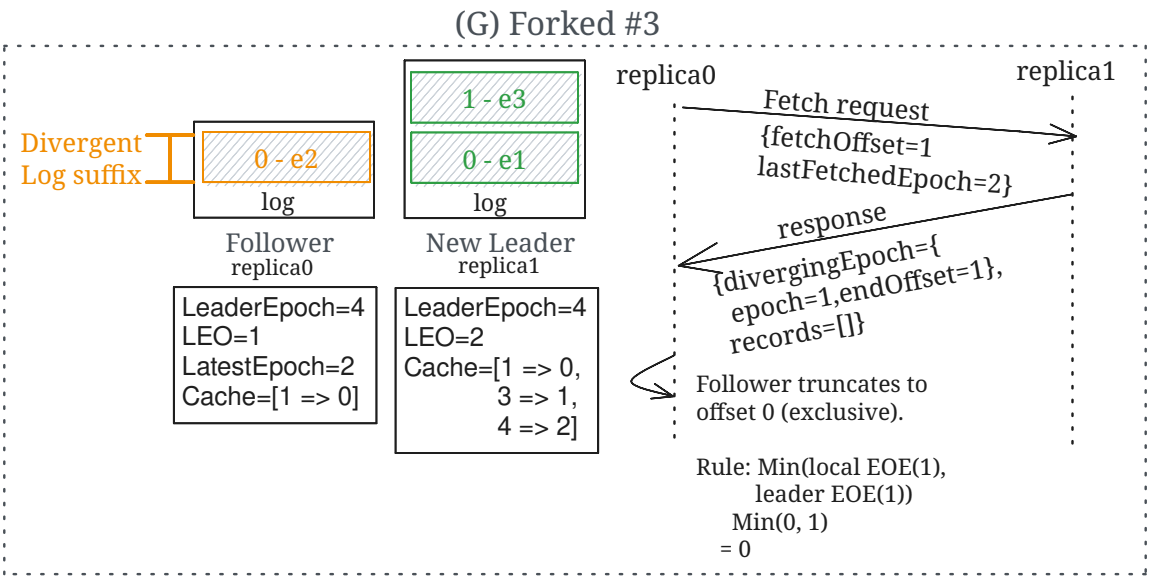

Fig 10. The follower log forks from the new leader log. The follower log and leader log have no common leader epochs at all.

The lastFetchEpoch does not exist in the leaders Start Epoch for Leader Epoch cache. It returns a diverging epoch with the highest epoch <= requested epoch (epoch 1) and the end offset as the start offset of the next highest epoch (offset 2).

The follower selects the minimum of:

- It's local End Offset for Epoch (EOE) for leader epoch 1 = offset 0. The follower does not have an epoch 1 or any lower epoch so uses offset 0.
- The leader's End Offset for Epoch for leader epoch 1 = offset 1

Truncation offset (exclusive) = 0. This completely truncates the log.

4. Partition reassignment (reconfiguration)

The protocol includes member reconfiguration (adding and removing of replicas) which is known as partition reassignment. A reassignment request is processed by the controller and a single request can contain reassignments for multiple partitions (across multiple topics).

The reassignment of any given partition can consist of:

- Only removing replicas, (lowering the replication factor (RF).

- Only adding replicas, increasing the replication factor.
- Both adding and removing replicas (moving/replacing replicas).

Two requests are involved in partition reassignments:

- An AlterPartitionReassignments request initiates (and potentially completes) a reassignment.
- An AlterPartition request sent by the leader replica can complete an ongoing reassignment.

4.1 General approach to different reassignments

4.1.1 Adding and removing (moves, replacements)

Reassignments that both add and remove replicas are performed in three phases:

1. Growth phase. A partition change is made to expand the replica set to be the union of the current and target assignment. The ISR is left unchanged.
2. Catch-up phase. The added replicas catch-up to the leader and get added to the ISR. This can span multiple AlterPartition requests from the leader if there are multple added replicas.
3. Complete (shrink/commit) phase. The final `adding` replica is added to the ISR by an AlterPartition request. As part of this request handling, the controller sets the replica set to the target assignment, excludes from the ISR and ELR any `removing` replicas, bumps the epochs and clears the `added` and `removed` sets and bumps the epochs.

The completion phase can only occur if the final ISR >= MinISR in order to respect the fault tolerance that MinISR is designed for.

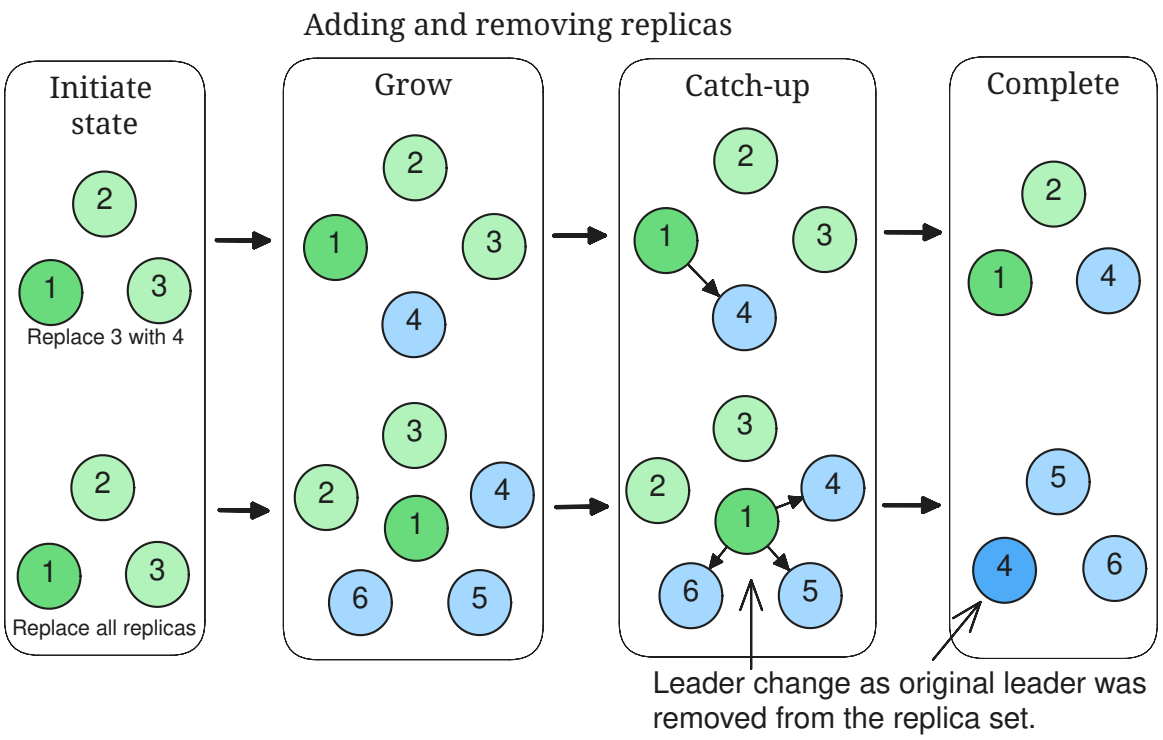

Fig 1. Partition reassignment consisting of adding and removing replicas.

4.1.2 Only removing (decreasing RF)

Reassignments that only shrink the replica set can be committed in a single partition change (as long as the final replica set and ISR would not be smaller than the MinISR).

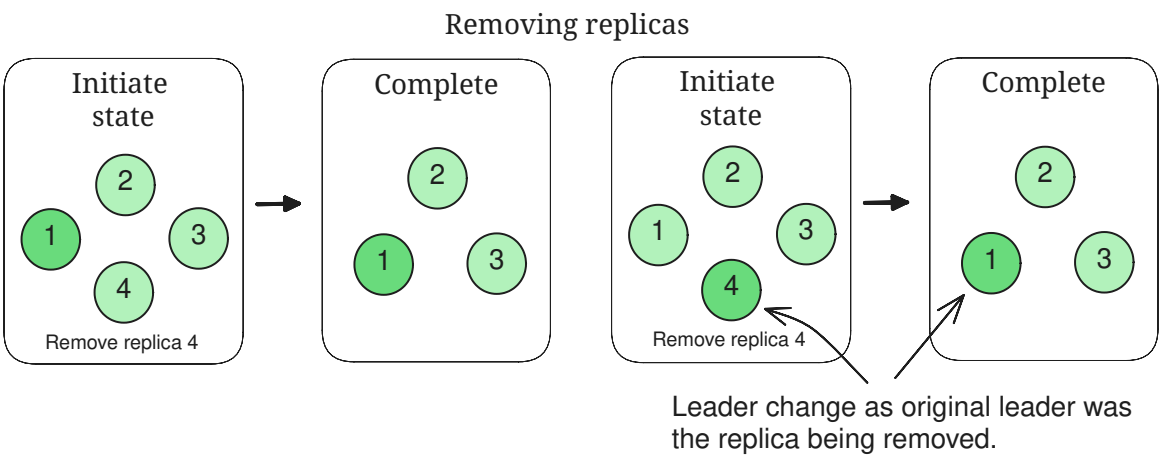

Fig 2. Partition reassignment consisting of only removing replicas.

4.1.3 Only adding (increasing RF)

Reassignments that only expand the replica set must be done across three phases:

1. Growth phase. Set the replica set to the target replica set.
2. Catch-up phase. Wait for all added replicas to be added to the ISR.

3. Complete phase. Clear the `added` and `removed` sets and bump the epochs.

Again, the AlterPartition requests perform the catch-up and complete phases. The completion can only occur if the final ISR is equal to or larger than the configured MinISR.

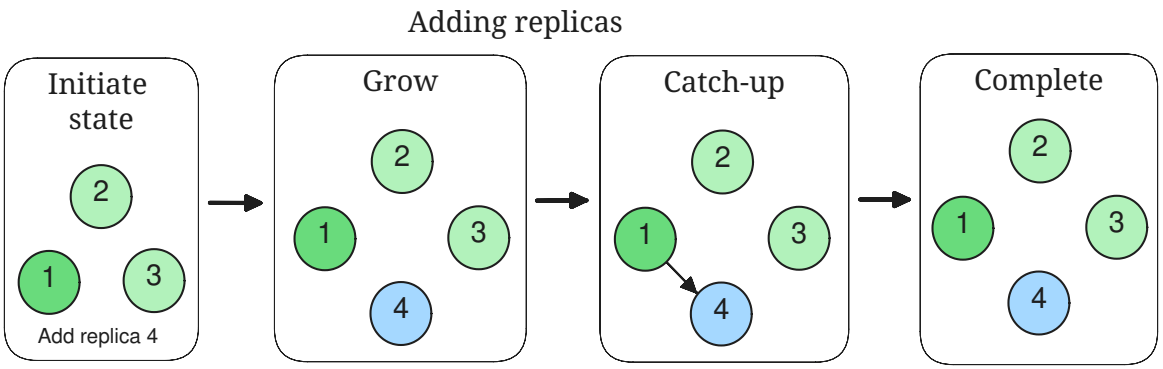

Fig 3. Partition reassignment consisting of only of adding replicas.

4.2 Controller logic

A partition has an ongoing reassignment if it has a non-empty `adding` or `removing` set in its metadata.

The controller can complete a reassignment either during the `AlterPartitionReassignments` request processing or during a later `AlterPartition` request processing. It may take just a single AlterPartition request to complete an ongoing reassignment or the reassignment may span multiple such requests. Only once the completion rules are satisfied, or a cancellation is issued does the reassignment terminate.

The controller can execute a reassignment in a single atomic partition change if no replicas are being added (just step 3 below). But when there are replicas to be added, it executes the reassignment as steps 1, 2 and 3.

1. Growth partition change:
   - Set the replica set to be the union of the current and target assignment.
   - Leave the leader, leader epoch and ISR unchanged.
   - Increment the partition epoch.
2. Catch-up partition change:
   - Add an `adding` replica to the ISR as a result of an AlterPartition request from the leader.
   - Potentially empty the ELR if ISR >= MinISR.
   - Increment the partition epoch.
3. Completion partition change:

    - Update the replica set to the target assignment.
    - Update the ISR and ELR to exclude any removed replicas.
    - Potentially empty the ELR if the updated ISR >= MinISR.
    - Change the leader if the leader is not a member of the target assignment (selecting from the new ISR).
    - Increment the leader epoch (whether the leader is changed or not).
    - Increment the partition epoch.

Note that partition changes 2 and 3 can occur in the same AlterPartition request processing.

Rules for a reassignment to be complete:

1. The final ISR if applied would be equal to or larger than the MinISR.
2. All `adding` replicas are members of the final ISR to be applied.

If the above rules are not met, then the reassignment remains ongoing and the final completion phase is not executed (the final replica set, ISR and ELR are not applied).

Because of the grow then shrink approach, a single reassignment does not need to have the current and target assignment intersect.

4.3 Partition replica logic

The replicas learn of the partition change via the metadata log replication.

- Newly added replicas become followers and start fetching from the leader from scratch (or receive the log as part of an AlterLogDir process).
- Removed replicas have their partition removed from its broker fetcher and it gets deleted locally.

The leader learns of the new replica set, it clears the state it maintains on any removed replicas (such as its LEO and broker epoch). When new followers catch-up, the leader sends AlterPartition requests to the controller.

4.4 Example reassignments

4.4.1 Example 1 - moving one replica

For example, due to an impaired broker 3, we want to move the replica on broker 3 to broker 4:

- current assignment: [1, 2, 3]

- target assignment: [1, 2, 4]

The controller takes the partition metadata through the following transitions:

1. Initial state:
    - Replicas=[1, 2, 3]
    - ISR=[1, 2]
    - Leader=1
    - LeaderEpoch=1
    - PartitionEpoch=2
    - Adding=[]
    - Removing=[]
2. The controller processes an AlterPartitionReassignments request to start the reassignment (grow):
    - Replicas=[1, 2, 3, 4]
    - ISR=[1, 2]
    - Leader=1
    - LeaderEpoch=1
    - PartitionEpoch=3
    - Adding=[4]
    - Removing=[3]
3. Replica 4 learns it is a follower and starts catching up to the leader.
4. The leader sends an AlterPartition request to add replica 4 to the ISR.
5. The controller receives the AlterPartition request and completes the reassignment:
    - (catch-up - intermediate state)
        - Replicas=[1, 2, 3, 4]
        - ISR=[1, 2, 4]
        - LeaderEpoch=1
        - PartitionEpoch=3
        - Adding=[4]
        - Removing=[3]
    - (complete - final state)
        - Replicas=[1, 2, 4]
        - ISR=[1, 2, 4]
        - LeaderEpoch=2
        - PartitionEpoch=4

    - Adding=[]
    - Removing=[]

Notice that because we only have one replica being added, the AlterPartition request processing emcompassed the catch-up and completion in one atomic partition change operation. If there were multiple replicas being added and multiple AlterPartition requests were sent by the leader, then the catch-up and completion could span multiple requests.

4.4.1 Example 2 - reducing replication factor

The administrators wish to reduce the replication factor from 5 to 3:

- Leader: 5
- Current assignment: [1, 2, 3, 4, 5]
- Target assignment: [1, 2, 3]
- MinISR=2

The controller takes the partition metadata through the following transitions:

1. Initial state:
    - Replicas=[1, 2, 3, 4, 5]
    - ISR=[4, 5]
    - Leader=5
    - LeaderEpoch=1
    - PartitionEpoch=2
    - Adding=[]
    - Removing=[]
2. The controller processes an AlterPartitionReassignments request to start the reassignment, however, the ISR consists of the two replicas we wish to remove. The final ISR would be empty, violating the MinISR rule. The reassignment is started but cannot complete yet:
    - Replicas=[1, 2, 3, 4, 5]
    - ISR=[4,5]
    - Leader=5
    - LeaderEpoch=1
    - PartitionEpoch=3
    - Adding=[]
    - Removing=[4, 5]
3. Replica 1 catches up and the leader sends an AlterPartition request to add replica 1 to

the ISR.

4. The controller receives the AlterPartition request and adds replica 1 to the ISR. However, the final ISR would consist only of replica 1, violating the MinISR rule. The reassignment remains ongoing:
    - Replicas=[1, 2, 3, 4, 5]
    - ISR=[1, 4, 5]
    - Leader=5
    - LeaderEpoch=1
    - PartitionEpoch=4
    - Adding=[]
    - Removing=[4, 5]
5. Replica 2 catches up and the leader sends an AlterPartition request to add replica 2 to the ISR.
6. The controller receives the AlterPartition request and adds replica 2 to the ISR. The final ISR [1, 2] meets the MinISR rule and the reassignment completes:
    - (catch-up - intermediate state)
        - Replicas=[1, 2, 3, 4, 5]
        - ISR=[1, 2, 5]
        - Leader=5
        - LeaderEpoch=1
        - PartitionEpoch=4
        - Adding=[]
        - Removing=[4, 5]
    - (complete - final state)
        - Replicas=[1, 2, 3]
        - ISR=[1, 2]
        - Leader=2
        - LeaderEpoch=2
        - PartitionEpoch=5
        - Adding=[]
        - Removing=[]

4.5 Canceling a reassignment

A reassignment may need to be canceled if an added replica is unable to catch-up. A cancellation request will revert the change.

However, there exists the possibility that not enough replicas of the original replica set are members of the ISR. In this case a revert is not safely possible as the final ISR would be smaller than the MinISR. If unclean leader election is enabled for the topic then the revert is completed anyway, by electing an ISR member as leader or if the ISR is empty, performing an unclean leader election. If unclean leader election is disabled (which is the common case) then the cancellation would abort with an error.

5. Asynchronous storage and recovery

Unlike protocols such as Raft and Paxos, the Kafka replication protocol is designed to support the asynchronous writing of log records to disk. This is attractive because disk flushes are expensive and even modern SSDs prefer large sequential writes than smaller random ones. By leveraging the page cache and the ability to perform fewer, larger writes to the underlying disk, Kafka gains a lot in sustained performance. However, the trade-off is that replicas can lose data that they previously acknowledged and without building special behavior into the protocol, local data loss can easily translate into global data loss.

Therefore Kafka employs recovery as a strategy to replace the stable storage guarantees it loses when not employing synchronous flushing. Other protocols such as Viewstamped Replication Revisited, Apache BookKeeper (without the journal) and the SAUCR paper have similar recovery mechanisms which allow them to write to disk asynchronously. However, because they are quorum based, the rules regarding leader eligibility are slightly different to Kafka.

Kafka can be configured with the following flush behavior:

- Every write (before HWM advancement and therefore writes are synchronous).
- Every nth write (allowing the HWM to advance in between flushes and therefore writes are asynchronous).
- Every n milliseconds (allowing the HWM to advance in between flushes and therefore writes are asynchronous).
- Only when a segment is rolled, split etc (asynchronous).

Note that the OS will be performing constant background flushes, keeping the dirty ratio below its configured threshold.

The rest of this section related to Kafka configured to write asynchronously to disk.

5.1 Recovery basics

5.1.1 Unclean shutdowns/abrupt terminations

Local data loss can occur when a broker experiences an unclean shutdown. We'll define a general unclean shutdown as just case 1 below but a lossy unclean shutdown requires both case 1 and 2:

1. The shutdown is either abrupt and instantaneous or does not leave enough time for the full shutdown sequence to occur.
2. The process page cache is also discarded.

A controlled shutdown sequence includes the flushing of all unflushed data to disk and writing its broker epoch to disk (if not present on start-up the unclean shutdown is detected). If this disk synchronization process is not performed and the page cache is also discarded then data loss can occur. The page cache would not be discarded if the Kafka process was terminated but the OS remained up. This is one advantage that Kafka has over Viewstamped Replication Revisited and Apache BookKeeper (without the journal) as both of those designs keep unflushed data in process memory, not the page cache.

5.1.2 General approach

Recovery is built on the strategy that unclean replicas should not be allowed to become leaders or leader candidates. Unclean replicas may have formerly been complete but now incomplete and so are treated as suspect until proven complete again through replication with the leader.

> Unclean Exclusion property: A replica that has restarted after an unclean shutdown must be excluded from leadership, the ISR and ELR until it has provably caught-up to the leader.

However, when the controller is unable to safely elect a leader for an unclean replica to recover from, a fallback strategy of physical log inspection is carried out to identify the best replica to elect as leader, from which any remaining unclean replicas can recover from.

Recovery is split into two phases:

1. Clean recovery which is possible when no more than a threshold of `f` replicas have experienced a concurrent lossy unclean shutdown (where `f = MinISR - 1`).
2. Unclean recovery is executed when clean recovery is not possible. Balanced-mode recovery offers the same safety guarantee as clean recovery but can also complete when `f >= MinISR` with probabilistic data loss.

5.2 Clean recovery via the ELR

Let's consider a replica affected by the unclean shutdown of its host broker. When the broker

starts (clean or unclean) it cannot complete its boot sequence until it has been fenced by the controller and have been removed from all partition leadership and all partition ISRs. This is a fundamental condition for a new registration. Under normal conditions, after the broker has registered, the replica is now an out-of-sync follower until it catches up with the leader and gets re-added to the ISR. All lost data is recovered and the partition returns to its normal level of redundancy.

However, there exist cases where the ISR shrinks to empty and in these cases there would be no leader to recover from. This is where the controller relies on the ELR to elect a leader which unclean replicas can then recover from. The MinISR is the minimum replication factor of committed records (bar broker failures). As long as no more than `MinISR - 1` brokers experience data loss there exists a copy of the committed data to recover from. The ISR can shrink to empty so to ensure MinISR durability, once the ISR reaches MinISR, any further ISR removals become moves to the ELR resulting in a combined ISR + ELR size of at least MinISR. The insight here is that when the ISR < MinISR, the HWM cannot advance so we can effectively freeze the leader candidate pool at this point.

During the controlled shutdown, after having flushed all unflushed partition data to disk, a broker writes its broker epoch to file. If the broker experiences an unclean shutdown, then this file is not written to disk. On start-up the broker registers with the controller using the broker epoch on file if it exists, else it uses an empty value. When the controller receives an empty broker epoch value in the broker registration request, the controller removes all replicas of that broker from their ELRs (if currently a member). These replicas would already have been removed from their respective ISRs when fenced but they could have remained members of their ELRs and therefore be leader candidates. By removing unclean replicas from ELRs we ensure no unclean replica can be a leader candidate.

Once registered and in the RUNNING status, all a broker's partition replicas are out-of-sync followers and can only be readded to their respective ISRs when they have become in-sync. This gives us the Unclean Exclusion property.

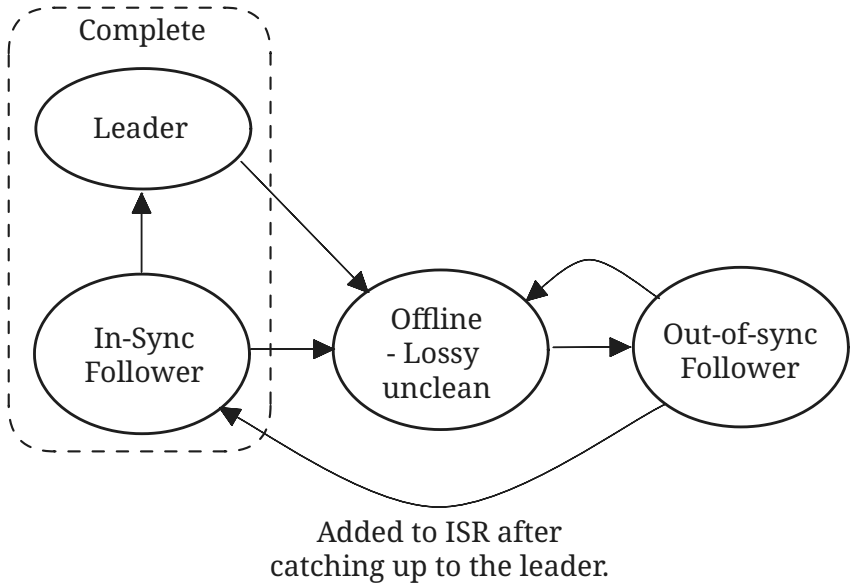

Fig 1. When a leader or in-sync follower experiences a lossy unclean shutdown, it must first pass through the out-of-sync phase where it is not a leader candidate. Only once it has proven itself to host the complete committed log can it become a leader candidate.

Out-of-sync followers are those that have become unreachable, fallen behind or become untrustworthy due to an unclean shutdown. The only way of becoming trusted again is to prove trustworthy by catching up to the leader.

5.2.1 ISR, ELR and replica completeness

The following figure describes how any given replica transitions between having and not having a completeness guarantee.

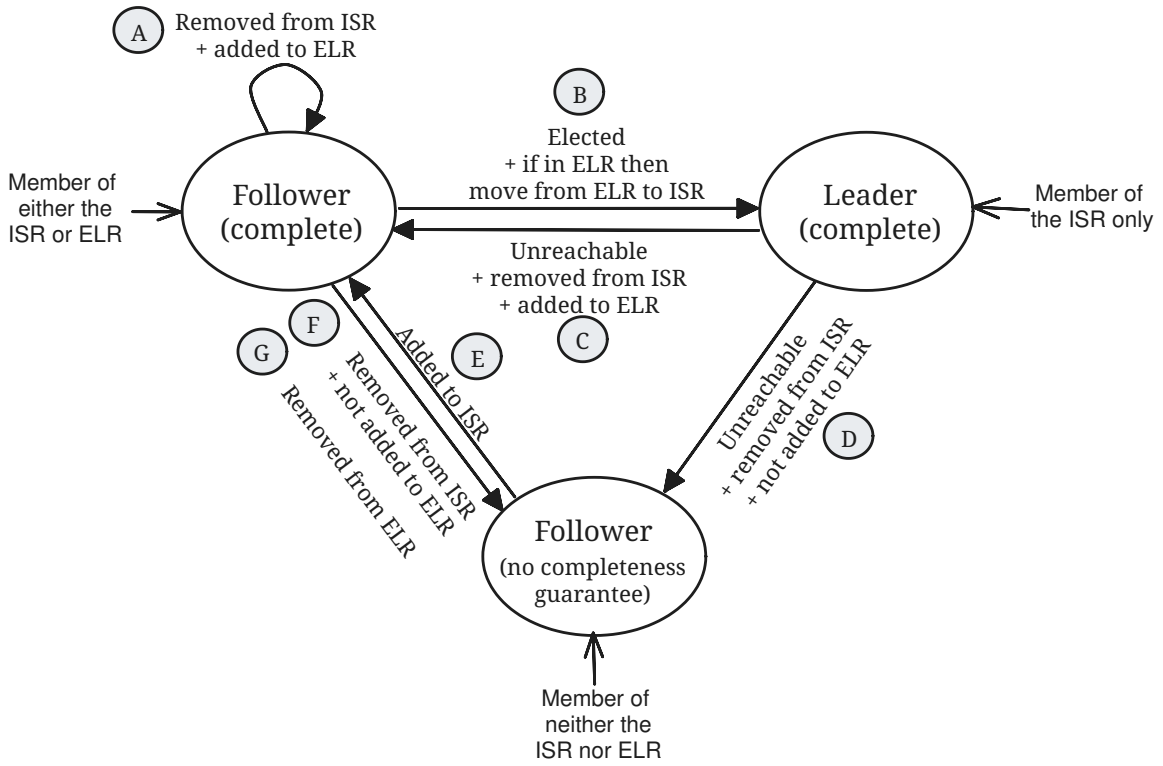

Fig 2. The ISR, ELR and unclean replicas.

While a replica itself doesn't get fenced (the broker does), for brevity replicas are referred to as getting fenced in the following transition descriptions. These transitions focus on a single

replica and its completeness guarantees:

- Case A. The controller fences replica R which is currently an in-sync follower and removes it from the ISR. Before committing this metadata change, the controller sees that the ISR is now < Min ISR therefore the controller adds replica R to the ELR. It is safe because the leader cannot advance the high watermark and so replica R will remain complete.
- Case B. The controller elects replica R, which is an unfenced follower and member of the ISR or ELR. The controller transfers the replica from the ELR to the ISR if it is currently an ELR member.
- Case C. The controller fences replica R, which the current leader. Another leader candidate is elected and replica R is removed from the ISR. Before committing this metadata change, the controller sees that the ISR is now < Min ISR therefore the controller adds the replica to the ELR. It is safe because the leader cannot advance the high watermark and replica R will remain complete.
- Case D. The controller fences replica R which is the current leader. Another leader candidate is elected and this replica is removed from the ISR. The new ISR >= Min ISR which means the new leader can advance the high watermark. Therefore replica R has no completeness guarantee and is not added to the ELR.
- Case E. Replica R, which is currently an out-of-sync follower, has caught up to the leader and the leader adds the out-of-sync follower to the ISR.
- Case F. Replica R, which is a currently an in-sync follower, goes out-of-sync and is removed from the ISR. The new ISR >= Min ISR therefore replica R loses its completeness guarantee and is not added to the ELR.
- Case G. Replica R is a fenced follower and a member of the ELR. The ISR just got expanded to be >= Min ISR and the high watermark can now advance again. Therefore replica R lose its completeness guarantee and must be removed from the ELR.

5.2.2 Clean ELR-based recovery example

The following is an example of how Kafka's clean recovery mechanism ensures data safety despite the lossy unclean shutdown of the leader and sole ISR member.

1. Initial state: `Leader=1, ISR=[1,2,3], ELR=[]`
2. Due to a degraded environment the ISR shrinks down to the leader (network connectivity has become flaky or some portion of the brokers have a temporary storage degradation for example).
    - First replica 2 is removed from the ISR. The ISR is still large enough for the HWM to advance therefore replica 2 is not added to the ELR: `Leader=1, ISR=[1,3], ELR=[]`

    - Then replica 3 is removed from the ISR. The ISR is now smaller than the MinISR and so replica 3 is added to the ELR. `Leader=1, ISR=[1], ELR=[3]` .
3. Replica 1, the leader and sole member of the ISR experiences an unclean shutdown. The controller fences replica 1 and elects replica 3 as the new leader. `Leader=3, ISR=[3], ELR=[]` .
4. Replica 1 reregisters, catches up to the leader and gets readded to the ISR. `Leader=3, ISR=[1,3], ELR=[]` .

No data loss (of acks=all requests) records could have occurred because as soon as the ISR shrank only to the leader the HWM could not have advanced and so replica 3 was guaranteed to be complete when it was later elected as leader.

5.3 Unclean recovery

If MinISR unclean shutdowns occur then the ISR and ELR of a partition can end up empty. When both the ISR and ELR are empty the controller does not know if any complete replicas exist or not and so cannot perform a clean election. However, unclean shutdowns do not always cause local data loss and so there may be complete replicas that could be eligible for leadership which the remaining replicas can recover from. Finding these replicas is the job of unclean recovery.

Unclean recovery has three strategies of different durability levels which are configured via `unclean.recovery.strategy` :

- `proactive` (optimized for availability).
- `balanced` (optimized for safety but automated)
- `manual` (optimized for users who have their own recourse that they wish to employ).

5.3.1 Modes

Proactive recovery triggers when a clean leader election is still possible but none of the candidates are available in that very moment. There may be ELR members but they are currently fenced. Instead of waiting for an ELR member to become unfenced, it immediately initiates the log inspection recovery. Once triggered, the proactive recovery process simply does a best-effort attempt to contact all replicas but picks the best log of any replies it gets within a short time period. This is for users who want availability above all.

Balanced recovery triggers only when the ISR and ELR have become empty and therefore a clean election is never going to be possible. It involves the controller inspecting the logs of the *LastKnownELR* and choosing the one with the highest epoch and offset. The LastKnownELR is a set stored in the partition metadata that contains previous ELR members that may still be complete. When an ELR member has an unclean shutdown, it is removed from ELR and

added to the LastKnownELR. The LastKnownELR is cleaned when the ISR reaches the MinISR and the HWM can advance.

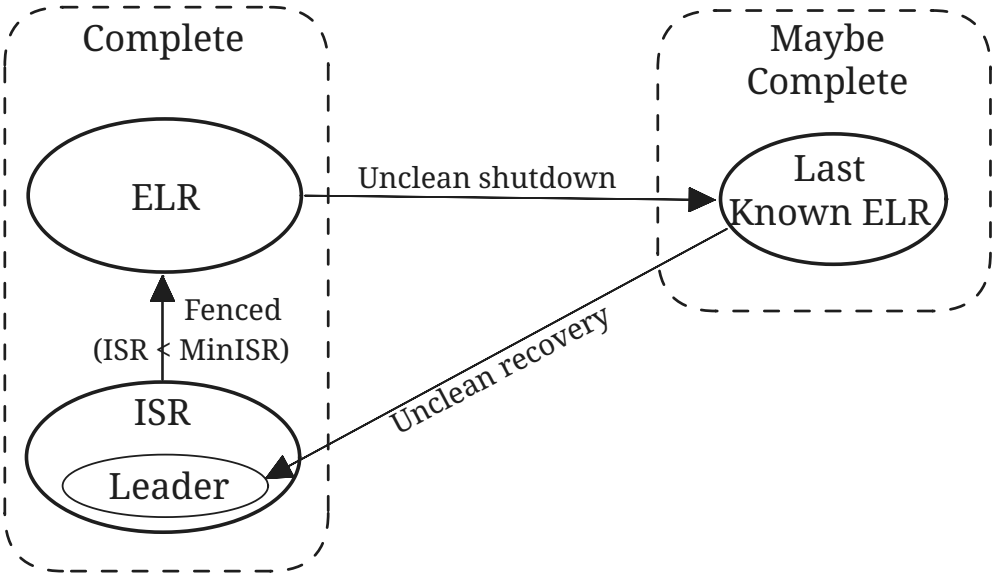

Fig 3. How replicas can move between the ISR, ELR and LastKnownELR.

5.3.2 Balanced-mode guarantees

The LastKnownELR constitutes the recovery quorum and is guaranteed to be MinISR in size when the ISR and ELR is empty.

> MinISR Recovery Quorum: The size of the unclean recovery quorum is guaranteed to be MinISR.

All LastKnownELR members are guaranteed to either be complete (because the unclean shutdown did not result in any data loss), or incomplete (because data loss did in fact occur). If there exists one LastKnownELR member that is complete then it is guaranteed to be selected as leader and moved to the ISR.

Therefore balanced unclean recovery has the same failure tolerance as the ELR-based clean elections: `f = MinISR - 1` where `f` is the number of lossy unclean shutdowns that should be tolerated without data loss.

The downside of unclean recovery is that the controller is unable to tell the difference between a data loss free unclean recovery and one where data loss did result. Therefore the controller must report all unclean recoveries as potential data loss events.

5.3.3 Balanced recovery example

The example uses the following configuration:

- `replication.factor=3`
- `min.insync.replica=2`
- `unclean.recovery.strategy=balanced`

1. Initial state:
    - `Leader=1, ISR=[1, 2, 3], ELR=[], LastKnownELR=[]`
    - `Replica 1: LEO=15, HWM=10`
    - `Replica 2: LEO=12, HWM=8`
    - `Replica 3: LEO=10, HWM=8`
2. A power outage takes down the cluster (causing an unclean shutdown on all brokers) and the cluster comes back online. The controller is a single-threaded state-machine and starts fencing the brokers one-by-one.
3. The controller fences broker 1 first and elects replica 2 as leader.
    - `Leader=2, ISR=[2, 3], ELR=[], LastKnownELR=[]`
4. The controller fences broker 2 and elects replica 3 as leader.
    - `Leader=3, ISR=[3], ELR=[2], LastKnownELR=[]`
5. The controller fences broker 3 and sets NoLeader.
    - `Leader=NoLeader, ISR=[], ELR=[2, 3], LastKnownELR=[]`
6. Broker 1 starts-up with loss of committed data and registers with the controller with an empty broker epoch value. It was not a member of the ELR so no partition metadata change is necessary.
    - `Leader=NoLeader, ISR=[], ELR=[2, 3], LastKnownELR=[]`
7. Broker 2 starts-up with *no* loss of committed data, registers with the controller with an empty broker epoch value and is moved from the ELR to the LastKnownELR.
    - `Leader=NoLeader, ISR=[], ELR=[3], LastKnownELR=[2]`
8. Broker 3 starts-up with loss of committed data, registers with the controller with an empty broker epoch value and is moved from the ELR to the LastKnownELR.
    - `Leader=NoLeader, ISR=[], ELR=[], LastKnownELR=[2, 3]`
9. The controller cannot perform clean election and triggers unclean recovery. It sends a GetReplicaLogInfo request to each replica in the LastKnownELR.
10. Once it has received a response from all the LastKnownELR replicas, it chooses replica 2 (of broker 2) as it had the log with the highest epoch and offset. In the end, no committed data was lost.

5.4 Deploying across failure-domains

Deploying distributed data systems across failure-domains enables the system to be reliable and available even under correlated failures such as a data center fire, a power surge that can

damage hardware, a power outage or some other incident that affects multiples servers at a time. Random failures such as disk drive failures are common but do not commonly affect multiple servers of the same cluster at the same time.

Kafka replication over multiple brokers enables a Kafka cluster to tolerate random failures and replication over brokers spread across multiple failure domains enables a cluster to tolerate correlated failures that might otherwise cause an unclean shutdown on multiple brokers. This is a standard deployment model for distributed data systems and it the recommended approach for Kafka. In the cloud this means deploying brokers across multiple availability zones.

6. Replication correctness

So far we've covered how the replicas communicate with each other and the controller but this is not enough to fully understand how the design guarantees consistency. Now we'll look at how the various epochs and behaviors support the internal properties which in turn support linearizability externally.

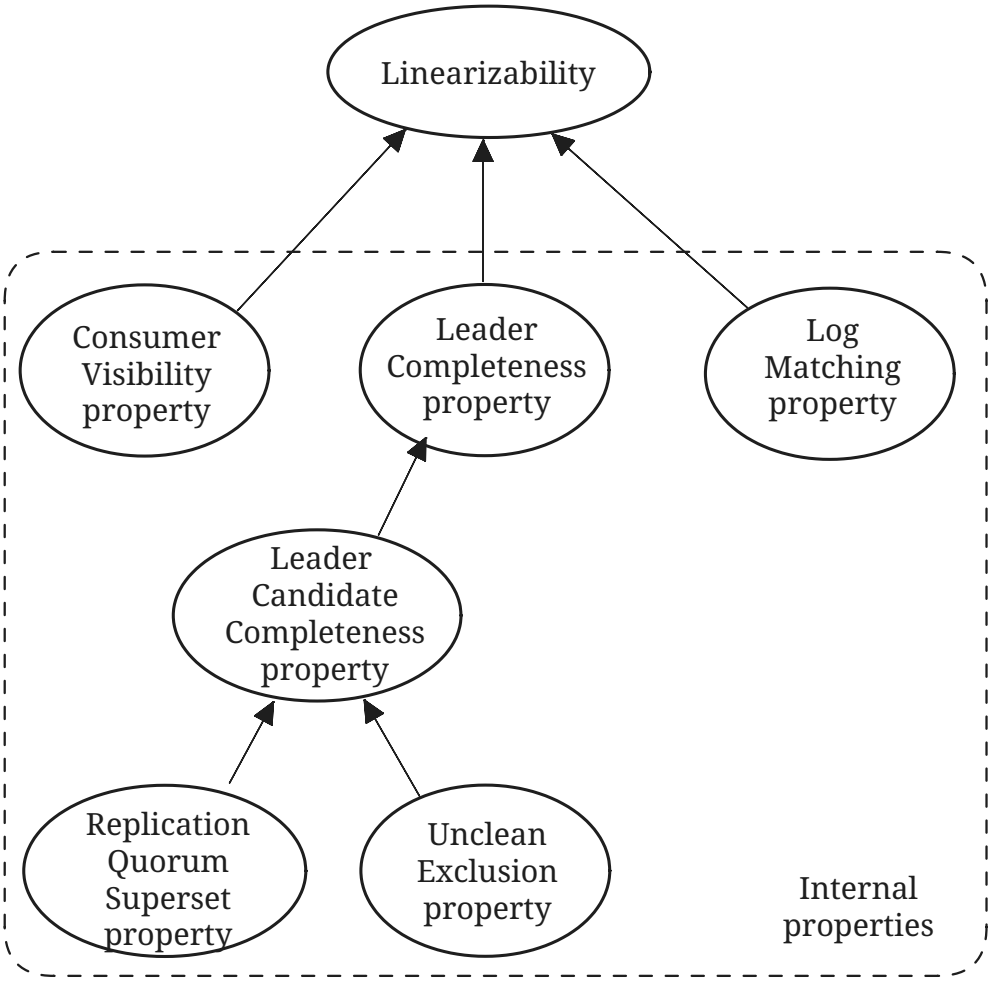

Fig 1. The relationships with the various internal properties to the linearizable semantics available to clients.

The two most important internal properties that are needed for consistency of the log itself are the Leader Completeness property and Log Matching property. If either of these two properties are violated the log itself is inconsistent. In turn we'll look at how the other more narrowly focused properties support Leader Completeness.

6.1 Linearizability Consistency

Kafka offers linearizable semantics to clients. To avoid duplication from the loss of producer acknowledgments, the wire protocol includes an idempotency mechanism based on counters, which matches the idempotency mechanisms of Raft (chapter 8) and Viewstamped Replication Revisited.

The definition of linearizability by jepsen.io is easily translatable to a log.

> Linearizability is one of the strongest single-object consistency models, and implies that every operation appears to take place atomically, in some order, consistent with the real-time ordering of those operations: e.g., if operation A completes before operation B begins, then B should logically take effect after A. https://jepsen.io/consistency/models /linearizable

Linearizability in Kafka is scoped to the partition (not individual records). For example, two non-overlapping writes to the same partition will result in `w1` being assigned a lower offset than `w2` , respecting the happens-before relationship of w1-before-w2. Likewise consumers will consume w1 and then w2 because w1 is earlier in the log than w2 and the log has a total order which is the same for all parties.

It is linearizable consistency rather than sequential because the advancement of the HWM triggers both positive producer acknowledgement and consumer visibility of records - and therefore qualifies for the recency condition of linearizability.

6.2 Baking-in unstable storage

The protocol needs to handle a limited number of lossy unclean shutdowns. Specifically, it should be able to avoid data loss with up to `Min ISR - 1` lossy unclean shutdowns.

A replica can get fenced and come back with a new broker epoch and only a prefix of its original log. The term unclean replica is used to denote a replica that has lost data after an unclean shutdown and has not yet recovered the data it had lost. Once a replica has recovered any lost data, it can be considered clean again.

The protocol should prevent unclean replicas from being elected as leader and prevent unclean replicas from joining the ISR. Subsequent sections will include discussion of unclean replicas where relevant.

6.3 Correctness of concurrent metadata changes

The partition metadata can be modified by the controller, the partition leader, a zombie controller or a zombie partition leader, all which can be attempting to change the metadata concurrently.

To prevent concurrent changes from causing damage there are a number of mechanisms at play:

1. The active-controller is the source-of-truth.
2. A controller must commit a change (via Raft) before it makes the change. Therefore a zombie controller cannot make changes.
3. The controller is single-threaded, processing one change at a time which means there is no possibility of race-conditions within the controller. This also means any active-controller initiated changes such as leader elections, ISR shrinks and reassignments are based on the current state.
4. The partition epoch of an AlterPartition request from a leader must match the current Partition Epoch in the metadata. This proves that the leader made its proposed change starting at a consistent view of the partition metadata - and therefore has no possible conflict. Note that an AlterPartition request can still contain stale broker-level data, this is discussed in section 6.5.

6.4 HWM monotonicity

The HWM stored on any given replica is the HWM that the replica knows of, not always what the “true” HWM is. The knowledge a replica has about the HWM may be arbitrarily stale and for short durations there may be no replica at all that knows the True HWM (even the leader).

The True HWM is monotonic and is the marker that truly separates committed from uncommitted records. However, the True HWM is not explicitly recorded anywhere (though a consumer offset can line-up with it).

The Local HWM on any given replica can be stale so treating the Local HWM as if it were the True HWM results in a non-monotonic value. The Local HWM itself is also not monotonic. The Local HWM on a follower is not monotonic because followers will overwrite their Local HWM with the HWM in a fetch response, even if the fetch response includes a lower HWM. This is possible because a leader election may choose a replica in the ISR that does not have the highest Local HWM (but does host the complete committed log).

However, the protocol itself requires HWM monotonicity for correctness and external systems (such as a stream processor like Flink) also want to be able to query the HWM and get monotonic results. The solution to this problem is discussed in the subsequent subsections.

6.4.1 Example of a lagging Local HWM

Here's an example of how a leader can end-up with a stale HWM:

1. Partition state: `Replicas=[r1, r2, r3], ISR=[r1, r2, r3], Leader=0` .
   - Replica 1 (leader): `LEO=0, HWM=0`
   - Replica 2: `LEO=0, HWM=0`
   - Replica 3: `LEO=0, HWM=0`
2. Leader receives a write with one record: Replica 0: `LEO=1, HWM=0` .
3. Replicas r2 and r3 send replica r1 a fetch request with `fetchOffset=0` and the leader responds with the one record it has.
   - Replica 1 (leader): `LEO=1, HWM=0`
   - Replica 2: `LEO=1, HWM=0`
   - Replica 3: `LEO=1, HWM=0`
4. Replicas 2 and 3 send replica 1 a fetch request with `fetchOffset=1` and so the leader now knows all replicas in the ISR have offset 0 and advances the HWM to 1 (remember it is exclusive). The true HWM is now also 1.
   - Replica 1 (leader): `LEO=1, HWM=1`
   - Replica 2: `LEO=1, HWM=0`
   - Replica 3: `LEO=1, HWM=0`
5. A consumer of group CG1 consumes offset 0 and commits its consumer offset to 1.
6. The controller fences replica 1 for some reason and elects replica 2. Replica 2 is the leader but has a HWM of 0 which is less than the True HWM of 1. Partition state: `Replicas=[r1, r2, r3], ISR=[r2,r3], Leader=1` .
   - Replica 1: `LEO=1, HWM=1`
   - Replica 2 (leader): `LEO=1, HWM=0`
   - Replica 3: `LEO=1, HWM=0`

At this point, the leader has a stale Local HWM.

6.4.2 "Applied HWM" monotonicity

While a leader may not always know the True HWM, a leader can determine it eventually. When a leader is elected, it may have a stale Local HWM though it cannot tell if its Local HWM is behind the True HWM or not. However, when it gets elected it sets its Leader Epoch Start Offset (LESO) to match its LEO. Once its Local HWM reaches the LESO it knows that the

Local HWM cannot be stale as it has been able to get all in-sync followers up to the current leader epoch. The Local HWM can only be higher than the LESO (and still within the bounds of the current leader epoch) because the leader itself advanced the HWM.

Therefore, all actions which require HWM monotonicity must only be performed once the leader has been able to advance its Local HWM to the LESO.

> Applied HWM monotonicity: All actions which are applied based on the Local HWM and which require a monotonic HWM value must wait until the leader has been able to verify it has reached the True HWM.

Example of actions that require Applied HWM monotonicity:

- Adding an out-of-sync follower to the ISR (more details in ISR expansion and Applied HWM Monotonicity).
- Responding to a ListOffsets request (which includes the current HWM in the response). If the leader does not yet know it has reached the True HWM it returns an “I don’t know” value for the HWM.

6.5 Correctness of leader elections

> Leader Candidate Completeness property: Every member of the ISR and ELR must host the complete committed log.

The ISR known to the leader acts as the quorum for replication (a record must be replicated to all members before acknowledgement). The ISR known to the controller plus the ELR forms the election quorum (or leader candidate pool). If the controller elects a leader from outside of the replication quorum then the new leader may not host the complete committed log and violate the Leader Completeness property which then leads to the global loss of committed data.

This issue is avoided by ensuring that all election quorum replicas are also members of the replication quorum (Leader Candidate Completeness). The following subsubsections discuss the details of how Leader Candidate Completeness holds, even under conditions such as lossy unclean shutdowns.

6.5.1 Broker fencing, broker epochs and unclean brokers

A broker epoch value denotes a single uptime session of a single broker. When a broker starts, it registers with the controller and receives a cluster-wide monotonic broker epoch integer. After having registered, it then starts to send periodic heartbeats to the controller. However, until the broker has been fenced by the controller, the controller will not accept a new registration attempt. For any given broker id, the previous broker epoch must be expired via fencing before a new epoch can be granted. Therefore the valid lifetime of a broker epoch

is from *broker registration* to *fencing + new registration*. Fencing alone does not terminate the validity of a broker epoch. A broker can get fenced due to a network partition and as soon as a heartbeat gets through to the controller, the broker gets unfenced with the same broker epoch.

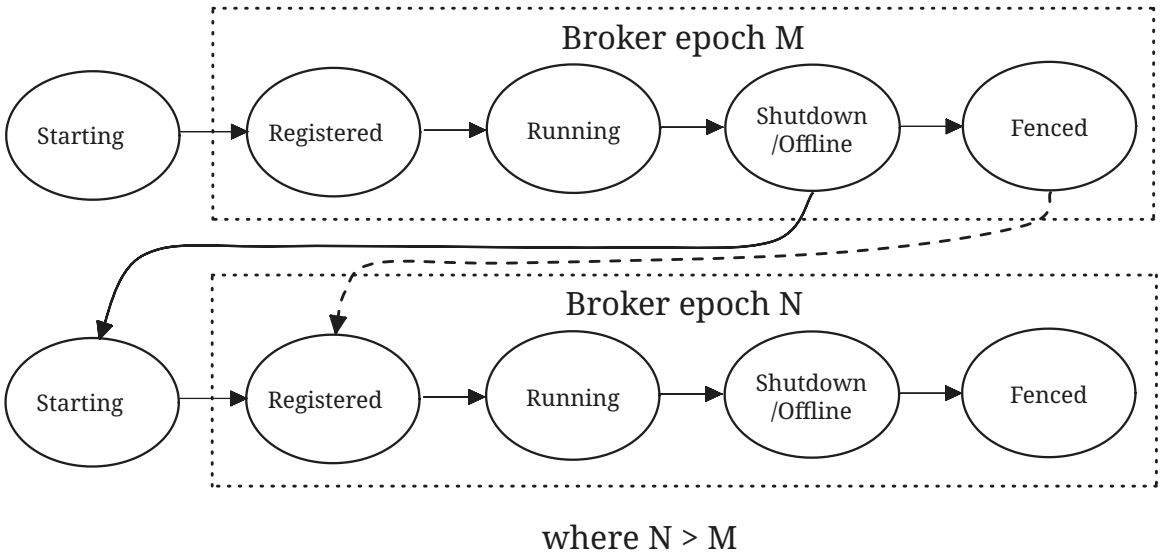

Fig 2. The previous broker epoch becomes invalid upon the assignment of a new broker epoch. This new broker epoch assignment can only occur once the broker (with its prior broker epoch) has been fenced. This prevents the ABA problem of quick broker reboots that afflicted the ZooKeeper based controller.

Why does a broker epoch denote a single uptime period rather than a single unfenced period? The reason is that the protocol needs to know about broker restarts in order to support the asynchronous storage engine. Only after a restart can a partition be online without data that it had previously confirmed (to either a client or a partition leader). Broker epochs are one important mechanism that the protocol uses to prevent lossy unclean shutdowns from turning local data loss into global data loss.

> Unclean Exclusion property: A replica that has restarted after an unclean shutdown must be excluded from leadership, the ISR and ELR until it has provably caught-up to the leader.

Let's consider some different cases where fencing and broker epochs are used for correctness when lossy unclean shutdowns occur:

- Case 1: The controller elects a replica as leader, but then the parent broker restarts uncleanly and the partition loses data. On the controller the broker is fenced, which

removes the replica from leadership and the ISR. During start-up, the broker registers with an empty broker epoch value (marking it as unclean) and the controler removes it from the ELR if it is a member. The replica is now an out-of-sync follower which becomes clean again by fetching from the new leader. It can only be readded to the ISR once it has caught up and is guaranteed to be complete (Unclean Exclusion property).

- Case 2: The leader successfully adds a replica to the ISR, but then the replica parent broker restarts uncleanly and the partition loses data. The rest is the same as Case 1.
- Case 3: The broker of an *out-of-sync* follower sends a fetch request but then restarts uncleanly and the partition loses data. The broker gets fenced but no change to the partition ISR is required (as the replica is not a member) so the partition epoch remains unchanged on the controller. Concurrently, the leader receives the fetch request from the now unclean replica and determines that the replica is now in-sync. The leader sends an AlterPartition request to the controller to add the replica (even though due to data loss its log is now lower than the HWM). Because this is an ISR expansion, the leader includes the broker epoch of the added follower. The controller rejects the AlterPartition request because the broker epoch of the added replica is lower than the new broker epoch. The now stale broker epoch is invalid because it was based on a state-of-the-world which may no longer be true. The broker epoch prevented an ISR change that would have violated the Unclean Exclusion property.

Cases 1 and 2 demonstrate how fencing protects the Unclean Exclusion property and Case 3 demonstrates how the broker epoch protects it. Without Unclean Exclusion, the Leader Candidate Completeness property would be violated.

6.5.2 The Maximal ISR

> Replication Quorum Superset property: The replication quorum used to advance the HWM is a superset of the election quorum.

Leader Candidate Completeness only holds if the replication quorum is a superset of the election quorum. The replication quorum is the ISR known to the leader and the election quorum is the ISR known to the controller plus the ELR. Both the controller and the leader replica share responsibility for ISR changes and both can have a stale view of it at any one time.

If the leader advanced its HWM based on a subset of the controller's election quorum then a leader election could cause data loss. For example, the controller believes the ISR to be `[r1, r2, r3]` but the leader (r1) believes the ISR to be `[r1, r2]`. The leader advances the HWM based on only `[r1, r2]` meaning that r3 could be lagging behind. The controller, believing that the ISR is `[r1, r2, r3]` could choose r3 as the new leader, despite it not hosting the complete committed log. As a result of r3 being elected, r1 and r2 would truncate their logs to match the new leader - losing committed, acknowledged records.

The Maximal ISR is used to ensure the Replication Quorum Superset property holds. When the leader replica sends an AlterPartition request to change the ISR, it sets its Maximal ISR to be the union of the Current ISR + Proposed ISR. The leader bases its HWM advancement on the fetch offset of its followers who are members of the Maximal ISR rather than the ISR.

For example:

- Expanding the ISR. The leader adds r3 to the ISR but the controller has not yet received the request and also could reject it.
  - Leader:
    - Current ISR = `[r1, r2]`
    - Proposed ISR = `[r1, r2, r3]`
    - Maximal ISR = `[r1, r2, r3]`
  - Controller: ISR = `[r1, r2]`
- Shrinking the ISR. The leader removes r3 from the ISR.
  - Leader:
    - Current ISR = `[r1, r2, r3]`
    - Proposed ISR = `[r1, r2]`
    - Maximal ISR = `[r1, r2, r3]`
  - Controller: ISR = `[r1, r2, r3]`

However, the Maximal ISR is not guaranteed to be a superset when the ISR is smaller than MinISR. The reason is that once the ISR < MinISR, the controller adds replicas to the ELR and the leader has no visibility of that.

For example:

1. All parties see `ISR=[1,2]` and the controller ELR is empty.
2. The controller fences broker 2.
   - Controller: `ISR=[1], ELR=[2]`
   - Leader: `ISR=[1], MaximalISR=[1]`
3. The leader sends an AlterPartition request to add replica 3 to the ISR. The request is inflight.
   - Controller: `ISR=[1], ELR=[2]`
   - Leader: `ISR=[1], MaximalISR=[1,3]`

At this point the replication quorum is not a superset of the election quorum. However, one of the necessary conditions for advancing the HWM is that the ISR (not Maximal ISR) known to the leader must also be equal to or larger than MinISR. Therefore while the Maximal ISR is not always a superset, it is guaranteed to be so when the HWM meets the conditions for advancement.

The relevant conditions for HWM advancement to respect the superset property are:

1. Fetch offset values of all followers in the Maximal ISR exceed the current HWM.
2. The size of the ISR is equal to or larger than MinISR.

If the controller accepts the AlterPartition request it will update the ISR, increment the partition epoch and send a success response with the new partition metadata. On receiving the success response, the leader commits the ISR change (also setting the Maximal ISR to the new ISR).

If the controller rejects the AlterPartition request, the leader rolls back the change to the last committed metadata version.

Let's look at what could happen **without** the eager application of the Maximal ISR:

1. The Leader is r1 and has the ISR of `[r1, r2]`. It has an empty log and the HWM is 0.
2. The leader receives a fetch request from r2 and r3 at offset 0 and responds with no records.
3. The leader sends an AlterPartition request with ISR `[r1, r2, r3]` to add r3 to the ISR. The leader does not apply the change yet, it instead waits for confirmation from the controller.
4. The leader receives some produce requests, writing offsets 0-9 to its log.
5. The leader receives a fetch request from r2 at fetch offset 0 and returns offsets 0-9 in the response.
6. The leader receives a fetch request from r2 at fetch offset 10 and advances the HWM to 10 (based on its current ISR of `[r1, r2]`).
7. The controller receives the AlterPartition request and sets the new ISR to `[r1, r2, r3]`. At this point r1 and r2 have offsets 0-9 and r3 has an empty log.
8. The controller performs a leader election and chooses r3 which does not have committed offsets 0-9.

During the period where the controller has applied the change but the leader has not received the response, the leader ISR is a subset of the controller election quorum and is now vulnerable to a leader change causing data loss - thus the need for the Maximal ISR.

6.5.3 ISR expansion and Applied HWM Monotonicity

In order for the leader to add a follower to the ISR, the follower must have reached the True HWM, else it would violate the Leader Candidate Completeness property. However, the leader only knows what the True HWM is once all in-sync followers have reached the Leader Epoch Start Offset (LESO). So we must apply ISR expansion to cases where either the leader's Local HWM >= True HWM or the fetch offset of the out-of-sync follower cannot be lower than the True HWM.

Therefore the leader adds another condition when determining if an out-of-sync follower is now in-sync (in addition to the latency rules):

1. The follower must have reached the Local HWM.
2. The follower must have reached the Leader Epoch Start Offset (LESO).

This works as follows:

- The Local HWM can only be stale when Local HWM < LESO.
- The Local HWM is guaranteed to be non-stale when Local HWM >= LESO.
- Therefore, the fetchOffset can only be equal to or higher than the Local HWM but also be less than the True HWM when it is also lower than the LESO. So condition 2 works to prevent that possibility.

6.5.4 ISR expansion and reassignments

The two conditions for adding a follower to the ISR that were discussed in the previous subsubsection are still not enough for correctness - only to ensure that the follower has reached the True HWM.

We must add a third condition that the leader must have received a fetch request in the current leader epoch from the follower in question. It is possible for a follower to have reached the LESO but not have sent a fetch request in the current leader epoch. For example, if we have consecutive leader changes but no new records are ever written to the log, the leader epoch advances but the LESO of each subsequent leader epoch remains the same.

The following illustrates the need for this third condition:

Reassignments can cause a replica to be removed then readded to a partition. Normally, the leader would learn of these changes in separate metadata fetches and so it would clear the follower state in the first update and then add follower state in the second. This follower state includes the LEO and broker epoch of the follower.

However, if this removal and addition happens quickly then the leader can end-up receiving both metadata updates in the same metadata fetch response. The partition does not react to each update separately, but to the final state of the partition after both updates have been applied to the current metadata. In this case the old state and new state both included the replica that was removed then added again, the only difference is that this replica may have previously been in the ISR and now is not. When the replica was removed, it’s on disk state was deleted, then it got readded as an empty replica. At this point, the new metadata only tells the leader that the replica got removed from the ISR, nothing more. Using only the first two conditions to assess if a follower is in-sync would result in this follower being added to the ISR again as its follower state indicates that its LEO has reached the HWM and LESO. Adding it now would violate the Leader Candidate Completeness property.

Therefore, we apply all of the following conditions for an out-of-sync follower to be classified as in-sync:

1. The follower must have reached the Local HWM.
2. The follower must have reached the Leader Epoch Start Offset (LESO).
3. The leader must have received a fetch request from the follower in the current leader epoch.

On top of that, for the HWM to advance we apply the conditions:

1. Fetch offset values of all followers in the MaximalISR exceed the current HWM.
2. The size of the ISR is equal to or larger than MinISR.

All these conditions ensure that Leader Candidate Completeness holds and therefore leader elections do not violate Leader Completeness (causing data loss).

6.6 Log matching

> Log Matching property: For all leader-follower pairs, the log prefix on both replicas is identical where the prefix is bounded by the min(Leader Local HWM, Follower Local HWM).

All writes go to the leader and are replicated to followers. This means that the order of log records is determined by the leader and all followers must match this same order. Once an offset is committed, it is stable which means it will survive leader changes. Any records at offsets equal to or greater than the HWM are unstable and may not survive leader changes. The Leader Candidate Completeness property dictates that all electable replicas host the complete committed log, however, each may have a different number of uncommitted offsets. After an election, a follower in the new leader epoch may have uncommitted records from a prior leader epoch which do not match the log of the leader. This was discussed at length in the Log Divergence section.

Divergence is detected by the leader via the first fetch request from a follower in the new leader epoch. The leader indicates the highest valid epoch and offset that it knows of that could match the follower's log. The follower then finds the highest epoch and offset that is lower or equal to the returned diverging epoch and offset then truncates its log to that point. After truncation, the *entire follower log* is guaranteed to match the prefix of the leader log. All it must do now is resume replication at this consistent position. Thus the Log Matching property holds.

6.7 Correctness of reassignments (reconfigurations)

Reassignments are comparatively simple in Kafka compared to the algorithms in Raft because

the membership is managed by controller. The controller itself is a single-threaded state machine on a Raft log and the metadata it hosts is the single-source of truth. Therefore reasoning about reassignments and inspecting reassignment progress is relatively simple.

The controller ensures that replica set changes do not cause data loss by the following rules:

1. The reassignment itself cannot leave the ISR smaller than MinISR.
2. The reassignment itself cannot add replicas to the ISR (which would violate Leader Candidate Completeness).
3. Durability is enhanced by performing reassignments in phases: grow, catch-up new replicas then shrink. This ensures redundancy levels are not reduced by reassignment itself, in fact reassignment increases redundancy, while new replicas catch-up to the leader. The leader itself notifies the controller, via AlterPartition requests to expand the ISR, when all new replicas contain the complete committed log. Only then can the shrink be performed which removes the old replicas from the replica set, the ISR and potentially the current leader from leadership.

6.8 Correctness of unclean recovery

Majority quorums are not effective when one or more nodes can go back on what they previously said. Majority quorums guarantee an overlap of one node but when a single node may have lost data it previously acknowledged the guaranteed overlap is lost.

Kafka avoids this problem by not requiring a majority quorum but tracking which replicas definitely were complete before the unclean shutdown(s). Additionally this group of replicas (LastKnownELR) is guaranteed to be MinISR in size. Therefore as long as all these replicas did not lose data then the controller is guaranteed to pick a replica without data loss. Thus unclean recovery ensures consistency when no more than the threshold of `f = MinISR - 1` replicas have lost data locally.

7. Configuring for availability over consistency

So far the focus has been on acks=all produce requests but Kafka can also be configured for availability over consistency. There are a number of tunable knobs available to the administrator and developer.

7.1 Produce requests with acks=1

The producer configuration `acks` simply allows the partition leader to positively acknowledge a produce request no matter the size of the ISR (even if Min ISR is set). This improves write latency and write availability.

Read availability is not impacted by the acks config though `acks=1` requests do benefit from the fact that HWM advancement is already geared more towards availability. The HWM advances based on the fetch positions of all followers in the ISR. If the ISR has three replicas then HWM advances to an offset when the fetch position has reached that offset in all ISR followers. But if the ISR has shrunk to the leader then the leader is free to advance the HWM based solely on itself. The result is that when using acks=1, the clients get end-to-end availability even when only the leader is functioning.

Note that `acks=1` records can be lost after leader elections as the producer is usually sent a positive acknowledgement while the record is still uncommitted. Uncommitted records have no guarantees of durability.

7.2 acks=all + min.insync.replicas=1 (Min ISR)

An alternative to `acks=1` is to use `acks=all + Min ISR=1`. This allows for the same write availability as `acks=1` but does not benefit from the write latency improvement. The reason for this is that `acks=1` allows the leader to acknowledge early, whereas `acks=all + Min ISR=1` allows the leader to accept writes when the size of the ISR is 1. If the ISR is of size 3, then the write must be replicated to all three before an acknowledgement is sent, whereas with `acks=1`, the leader would have sent the acknowledgement as soon as it had written it to its log.

`acks=all` requests survive leader elections* so we get an improvement to safety without compromising on availability. The operator can boost availability further by timing cluster rolls to when no partitions have degraded ISRs. If the cluster roll is initiated when all partitions have full ISRs then partitions have leader candidates to fail-over to during the rolls.

* note that safety is still impaired compared to `Min ISR=2` as the safety tolerance for lost brokers is `Min ISR - 1` (but this section is focused on availability over consistency).

7.3 Unclean leader elections

If a partition gets itself into a Last Replica Standing situation and this last replica (the leader) goes offline the partition remains unavailable until that specific replica becomes functional again. To avoid this unavailability the topic can be configured to allow unclean elections which allows a functional replica outside of the ISR to be elected. This can result in data loss but keeps the partition available.

Unclean elections have consequences such as:

- True HWM is no longer monotonic.
- Log divergence truncation may need to go through several rounds before the divergence is resolved. However, eventually, logs will still converge.
- Consumers may need to reset their offsets (because the True HWM may have gone backwards).

There may be additional logic in the replicas that has been missed by this protocol description that is required to handle the impacts of unclean leader elections. This is future work.

8. Broker lifecycle

So far broker restarts, fencing and broker epochs have been discussed but the full start-up and shutdown sequence has not been described. This section is not critical to understanding the protocol but does provide more detail that can help with a general understanding of Kafka.

8.1 The broker start-up sequence

A broker cannot simply start-up, activate any replicas it hosts and let the replication fly. There is a start-up sequence a broker goes through which starts with broker registration where the broker is assigned a monotonic broker epoch and is completed via piggy-backing information on broker heartbeat requests and responses.

The primary role of heartbeats is as a failure detection mechanism for the control plane. However, with the use of some additional fields in the heartbeat request and response, the controller and broker can communicate intent such as a controlled shutdowns, or status information such as whether a broker is fenced or not.

When a broker boots, it begins in the STARTING state. It sends a BrokerRegistration request to the controller with a unique randomly generated “incarnation id” which it stores in memory. The controller on receiving this BrokerRegistration will either accept it or reject it. It will accept the request if one of the following is true:

- The controller has no stored registration for this broker.
- The broker has a stored registration but this registration has expired (fenced due to lack of heartbeats).
- The broker has a non-expired registration which has an identical incarnation id. This is to ensure that a broker that is sending registration retries is able to complete registration. The incarnation id plays no further role in the broker lifecycle and no role at all in Kafka replication protocol.

When the controller accepts the BrokerRegistration request, it appends a BrokerRegistration record to the metadata Raft log and creates/updates in-memory state about the broker.

Key state maintained about a broker are:

- `broker epoch` (set to the offset of the registration record in the metadata log which is guaranteed to be unique and monotonic).
- `registration offset` (the offset of a broker's registration record).
- `fenced` (true/false)
- `shuttingDown` (true/false)
- `shouldShutdown` (true/false)

As previously discussed, a new broker epoch can only be assigned when the broker has been fenced. The lifetime of a broker epoch is bounded by a final fencing plus a new registration (and new epoch).

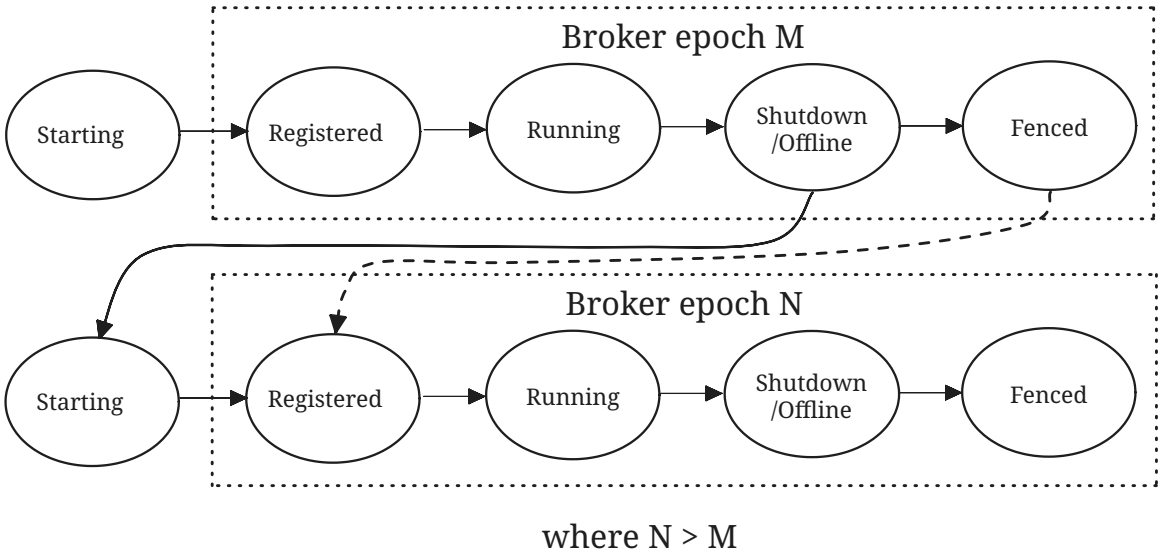

Fig 1. A broker epoch lifetime.

Upon receiving a successful response to its BrokerRegistration request, the broker switches to RECOVERY which is a local broker process where the broker's storage layer takes stock of its current condition. This process is not related to the replication protocol - it is purely for internal housekeeping. Meanwhile, two other things occur. It starts the pull-based metadata log replication process where it fetches committed metadata update records from the controller. Secondly, it starts to send periodic heartbeats to the controller.

Each heartbeat includes a field `wantFence` which indicates whether the broker wants to be fenced or not. At first, the broker wants to be fenced but once the storage layer has completed its start-up and the broker has caught-up to its own BrokerRegistration record in the metadata log, it sets the `wantFence` field to false in its heartbeats. On receiving a heartbeat request with `wantFence=false`, the controller unfences the broker (checking again that its metadataOffset has reached its broker registration offset). It lets the broker know it is unfenced via the field `isFenced=false` in the heartbeat response.

The reason why a broker waits until it has caught up to its own BrokerRegistration record in the metadata log is so that it doesn't start trying to replicate data based on a very stale copy of the cluster metadata. Waiting until it reaches its own registration record makes sure it is very close to the controller's view of the cluster. Note that this is just an optimization and not required for correctness. The fetch mechanism uses various epochs to prevent stale replicas from causing issues.

The broker now switches to the RUNNING state and all its partition replicas assume their role as either leader or follower based on the currently known cluster metadata. This metadata can be slightly stale but the various epochs prevent a stale replica from making progress.

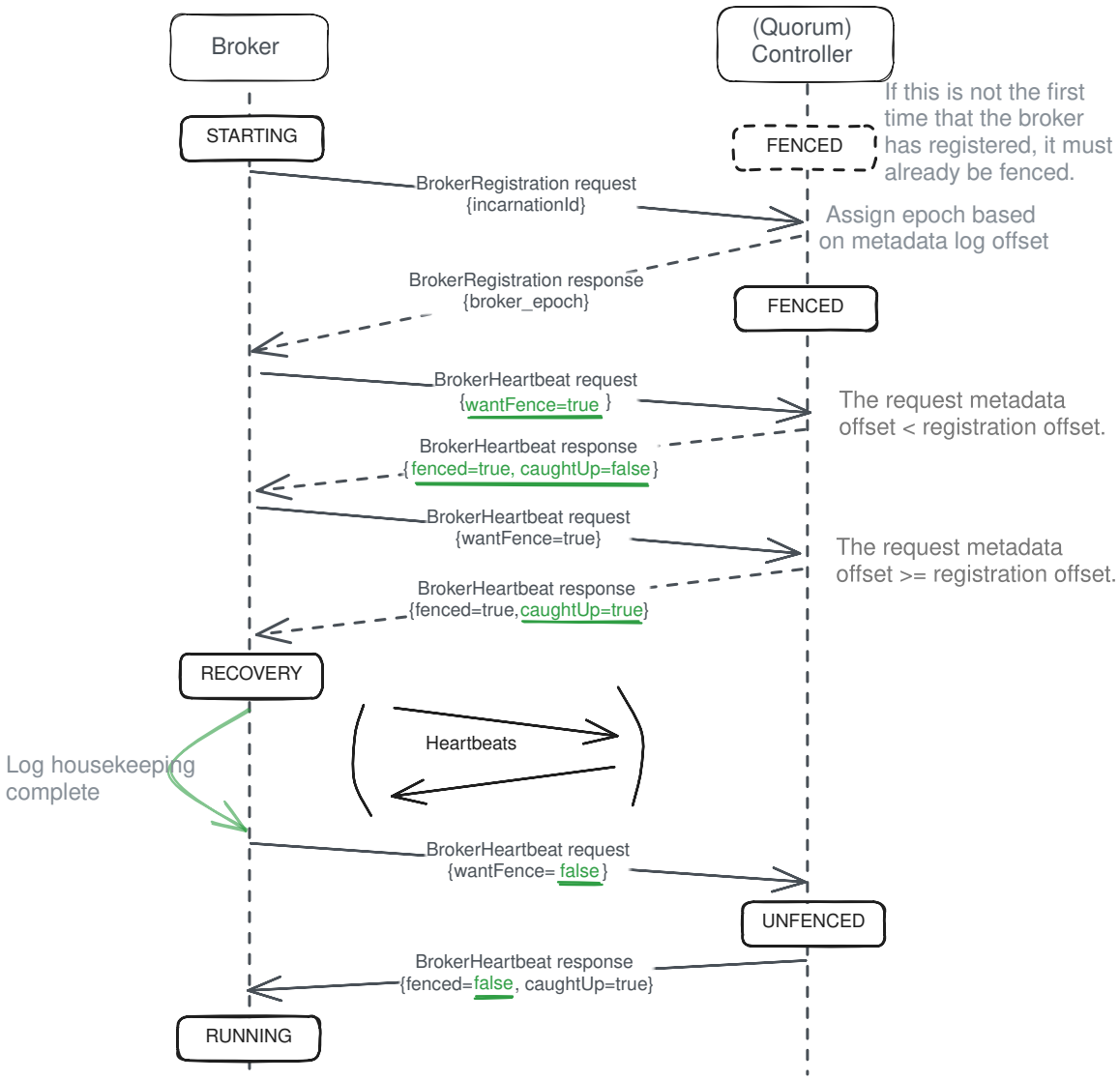

Fig 2. The broker start-up sequence.

8.2 Steady state

A broker will remain in the RUNNING state until it shuts down or is abruptly terminated. While in the RUNNING state, the controller can fence and unfence the broker based on heartbeats and lack of heartbeats. As long as the broker is still running, eventually it will send a heartbeat to the controller and the controller will unfence it. Unfencing does not increment the broker epoch (that only occurs during broker registration when a broker starts up).

8.3 The controlled shutdown sequence

When a broker has been told to shutdown, it switches to the PENDING_CONTROLLED_SHUTDOWN state and communicates its intent to the controller via a heartbeat with `wantShutdown=true` set in its heartbeat. On receiving this request the controller records this intent in the broker registration and checks if the broker is a leader of any partitions. If the broker hosts any leader replicas then the controller places the broker in the `CONTROLLED_SHUTDOWN{fenced=false, shouldShutdown=false}` state and it records the current end offset of the metadata log as the controlledShutdownOffset. If the broker has no leader replicas it places the broker in the `SHUTDOWN_NOW{fenced=true, shouldShutdown=true}` state which will allow the broker to immediately shutdown.

The controller only allows the broker to switch from CONTROLLED_SHUTDOWN to SHUTDOWN_NOW once two things happen:

- The broker hosts no leader replicas.
- All other brokers which are unfenced and not shutting down have reached the `controlledShutdownOffset` and therefore have received the metadata updates related to the unloading of partition leadership of the shutting down broker.

Heartbeats continue until the broker receives a response with `shouldShutdown=true`.

The controlled shutdown sequence exists to ensure that all affected partitions have continued availability by ensuring that other live brokers have taken over leadership before the broker shuts down.